Personal Information

Name: **Buğra Sipahioğlu**

StudentID: **14318334**

Email: **bugra.sipahioglu@student.uva.nl**

Submitted on: **23.03.2026**

Data Context

This study draws on two complementary datasets from the domains of event extraction and computational narratology. The first is MAVEN (MAssive eVENt detection dataset; Wang et al., 2020), a large-scale, document-level event detection benchmark derived from encyclopedic Wikipedia articles covering real-world events. MAVEN provides a fine-grained ontology of over 160 event types with annotated event triggers and negative triggers across 4,480 documents, and serves as the supervised training resource for the event extraction backbone (BERT+CRF) used in our pipeline. The second is Tell Me Again! (Hatzel & Biemann, 2024), a multilingual narrative similarity corpus containing approximately 29,505 unique narrative works and over 109,000 independently written plot summaries collected from Wikipedia in five languages (English, German, French, Italian, and Spanish). Summaries of the same underlying work form relevance clusters, which enables us to conduct retrieval-based evaluation of narrative similarity.

Together, these datasets support a two-stage research design: MAVEN enables robust, supervised extraction of structured event representations from narrative text, while Tell Me Again! provides the evaluation framework for measuring whether explicitly modeling temporal and causal relations between those events improves the retrieval of narratively similar stories beyond event-only and implicit dense embedding approaches. Both datasets are publicly available, Wikipedia-derived resources, though they differ in domain focus -- MAVEN covers factual, encyclopedic event descriptions whereas Tell Me Again! spans fictional plot retellings that vary widely in length, detail, and abstraction.

Data Description

The exploratory data analysis is organized into two parts, one for each dataset.

MAVEN. The corpus contains 4,498 Wikipedia-derived documents split into train (2,913), validation (710), and test (857). Each document is a JSON object with a title, unique ID, tokenized sentence content, a list of annotated events (each with a type drawn from a 168-class ontology and one or more trigger-span mentions), and a list

of negative triggers. No missing values are present in any split, and train and validation exhibit closely matched distributions across all document-level statistics: the mean document length is approximately 11 sentences (~286 tokens), with right-skewed tails extending to substantially longer documents. Vocabulary size and hapax counts similarly parallel between splits, confirming that the validation set is a representative sample of the training distribution. At the univariate level, the event-type distribution follows a pronounced long-tail pattern: the top three types (Causation, Process_start, Attack) each account for roughly 3.4--3.6% of all trigger mentions, while the bottom of the ontology contains many rare types. The corpus is heavily class-imbalanced at the trigger-detection level, with approximately 80.6% of annotated tokens being negative triggers (P(negative)=0.806). Thus, a trivial "always predict negative" baseline achieves 80.6% accuracy, and the majority-type baseline yields only 3.6% accuracy for event-type classification. For our pipeline, we rely on the BERT+CRF model reported by Wang et al. (2020), which achieves 67.8 F1 on the MAVEN trigger-classification benchmark, and we do not aim to improve upon MAVEN event detection itself.

Tell Me Again! The corpus comprises 29,505 unique narrative works with a total of 109,691 raw summaries in five languages (German: 28,947; English: 26,385; French: 21,862; Italian: 19,737; Spanish: 12,760). To create different versions of the same stories, technical translation to English language has been applied, making multiple English versions of summaries that belong to same work. However, some of the translations were almost identical to each other, which makes the retrieval task unefficient. Thus, they removed near-duplicate summaries of the same work (i.e., cosine similarity between them exceeds the threshold of 0.6), making the length of the dataset 97,813 summaries long. Summary lengths differ substantially across languages: English, German, and French summaries are comparably long, while Italian and Spanish tend to be shorter. Deduplication produces 423 singleton clusters (works with only one usable summary), which must be excluded from retrieval evaluation. A series of data-quality inspections revealed: 1,112 stories lack genre metadata (relevant if stratified sampling by genre is needed); 3,120 stories lack a native English summary, but 2,698 of those still have at least 2 English summaries from machine translation and are therefore pipeline-eligible; 1 story has no translated summaries at all and should be excluded; and 32 stories have blank Wikidata descriptions (not critical for our pipeline). After applying pipeline-eligibility criteria (≥2 deduplicated English summaries and at least one translated summary), 29,082 works remain (98.6% of the full corpus), of which 90.7% possess a native English summary and 9.3% rely solely on translations.

A context-window analysis using the Llama-3-8B-Instruct tokenizer shows that per-summary token counts have a right-skewed distribution (mean ≈ 1,509, median = 1,250, std ≈ 1,222). Only 14 summaries (from 14 distinct stories) exceed the 8,192-token context limit; after removing these outlying summaries, 11 of the 14 affected stories still retain ≥2 usable summaries and remain viable for retrieval evaluation. A split-representation analysis using the official train/dev/test partitions confirms that these splits are representative of the full pipeline-eligible corpus with respect to

summary length distributions, number of genres, and genre-type proportions. Finally, the SimilarityDataset -- which uses the same official train/dev/test split to construct positive/negative summary pairs for similarity evaluation -- is perfectly class-balanced across all splits: 50% positive (same-story) and 50% negative (different-story) pairs (train: 209,160 pairs; dev: 26,160; test: 26,196), which yields a majority-class baseline accuracy of 50.00% that any similarity model must exceed.

In [1]:
```
# Imports
import os
import numpy as np
import pandas as pd
from tell_me_again import StoryDataset, SimilarityDataset
import json
import seaborn as sns
import matplotlib.pyplot as plt
from nltk.probability import FreqDist
import nltk
from pprint import pprint
from transformers import AutoTokenizer
from tqdm import tqdm
import zipfile
import seaborn as sns

# Download package(s)
nltk.download("punkt_tab", quiet=True)
```

```
/Users/bugrasipahioglu/repo/uva-thesis-computational-narrative-analysis/ve
nv/lib/python3.14/site-packages/tqdm/auto.py:21: TqdmWarning: IProgress no
t found. Please update jupyter and ipywidgets. See https://ipywidgets.read
thedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
PyTorch was not found. Models won't be available and only tokenizers, conf
iguration and file/data utilities can be used.
```

Out[1]: True

Dataset 1: MAVEN

Data Loading

In [2]:
```
# Define the core directories
PROJECT_ROOT = os.path.abspath(os.path.join(os.path.dirname("."), ".."))
MAVEN_DIR_RAW = os.path.join(PROJECT_ROOT, "data", "raw", "MAVEN")

# Load the data
df_train = pd.read_json(os.path.join(MAVEN_DIR_RAW, "train.jsonl"), lines
df_val = pd.read_json(os.path.join(MAVEN_DIR_RAW, "valid.jsonl"), lines=T
df_test = pd.read_json(os.path.join(MAVEN_DIR_RAW, "test.jsonl"), lines=T
```

Analysis 1: Corpus-level Overview

We inspect corpus size (number of documents and variables per split), variable types, and missing values. We also show one example document so the structure of content, events, and negative_triggers is clear.

In [3]:
```
# Analyse the shapes of train, valid, and test splits
print("\033[1;33m\nShapes of the splits:\033[0m")
for name, df in [("Train", df_train), ("Valid", df_val), ("Test", df_test
    print(f"  {name}: {df.shape}")

# Print the variable types and non-null counts for all splits
print("\033[1;33m\nVariable types and non-null counts (train):\033[0m");
print("\033[1;33m\nVariable types and non-null counts (validation):\033[0
print("\033[1;33m\nVariable types and non-null counts (test):\033[0m"); p

# Print the missing values per variable
print("\033[1;33m\nMissing values per variable (train):\033[0m\n" + str(d
print("\033[1;33m\nMissing values per variable (validation):\033[0m\n" +
print("\033[1;33m\nMissing values per variable (test):\033[0m\n" + str(df

# Print one full document entity from the train split for inspection
print("\033[1;33m\nExample document from train split (truncated):\033[0m"
sample_doc = df_train.iloc[0].to_dict()
# Limit the example to 3 key fields (e.g., 'title', 'id', and first 1 sen
limited_doc = {k: sample_doc[k] for k in ['id', 'title'] if k in sample_d
if "content" in sample_doc:
    # Show only first sentence's tokens
    limited_doc["content"] = sample_doc["content"][:1]
if "events" in sample_doc:
    limited_doc["events"] = sample_doc["events"][:1]
if "negative_triggers" in sample_doc:
    limited_doc["negative_triggers"] = sample_doc["negative_triggers"][:1
print(json.dumps(limited_doc, indent=2, ensure_ascii=False))

# Print the number of event types in the corpus
# Count unique event types in the corpus (across train, valid, and test s
def get_event_types(df):
    event_types = set()
    if "events" in df.columns:
        for events in df["events"]:
            if isinstance(events, list):
                for event in events:
                    etype = event.get("type")
                    if isinstance(etype, str):
                        event_types.add(etype)
    return event_types

event_types_train = get_event_types(df_train)
event_types_val = get_event_types(df_val)
event_types_test = get_event_types(df_test)

all_event_types = event_types_train.union(event_types_val).union(event_ty
print(f"\033[1;33m\nNumber of unique event types in the corpus (across al
```

```
Shapes of the splits:
  Train: (2913, 5)
  Valid: (710, 5)
  Test: (857, 4)

Variable types and non-null counts (train):
<class 'pandas.DataFrame'>
RangeIndex: 2913 entries, 0 to 2912
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   title              2913 non-null   str
 1   id                 2913 non-null   str
 2   content            2913 non-null   object
 3   events             2913 non-null   object
 4   negative_triggers  2913 non-null   object
dtypes: object(3), str(2)
memory usage: 113.9+ KB
None

Variable types and non-null counts (validation):
<class 'pandas.DataFrame'>
RangeIndex: 710 entries, 0 to 709
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   title              710 non-null    str
 1   id                 710 non-null    str
 2   content            710 non-null    object
 3   events             710 non-null    object
 4   negative_triggers  710 non-null    object
dtypes: object(3), str(2)
memory usage: 27.9+ KB
None

Variable types and non-null counts (test):
<class 'pandas.DataFrame'>
RangeIndex: 857 entries, 0 to 856
Data columns (total 4 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   title       857 non-null    str
 1   id          857 non-null    str
 2   content     857 non-null    object
 3   candidates  857 non-null    object
dtypes: object(2), str(2)
memory usage: 26.9+ KB
None

Missing values per variable (train):
title                0
id                   0
content              0
events               0
negative_triggers    0
dtype: int64

Missing values per variable (validation):
title                0
```

```
id                   0
content              0
events               0
negative_triggers    0
dtype: int64
```

Missing values per variable (test):

```
title         0
id            0
content       0
candidates    0
dtype: int64
```

Example document from train split (truncated):


```
{
  "id": "8307a6b61b84d4eea42c1dd5e6e2cdba",
  "title": "2006 Pangandaran earthquake and tsunami",
  "content": [
    {
      "sentence": "The 2006 Pangandaran earthquake and tsunami occurred on July 17 at along a subduction zone off the coast of west and central Java, a large and densely populated island in the Indonesian archipelago.",
      "tokens": [
        "The",
        "2006",
        "Pangandaran",
        "earthquake",
        "and",
        "tsunami",
        "occurred",
        "on",
        "July",
        "17",
        "at",
        "along",
        "a",
        "subduction",
        "zone",
        "off",
        "the",
        "coast",
        "of",
        "west",
        "and",
        "central",
        "Java",
        ",",
        "a",
        "large",
        "and",
        "densely",
        "populated",
        "island",
        "in",
        "the",
        "Indonesian",
        "archipelago",
        "."
      ]
    }
```

```
    ],
    "events": [
      {
        "id": "40b3b20bc2eeb6b163538b82c1379ead",
        "type": "Know",
        "type_id": 1,
        "mention": [
          {
            "trigger_word": "observed",
            "sent_id": 5,
            "offset": [
              12,
              13
            ],
            "id": "7fcf445a679aa13511278d321a908bd2"
          }
        ]
      }
    ],
    "negative_triggers": [
      {
        "trigger_word": "populated",
        "sent_id": 0,
        "offset": [
          28,
          29
        ],
        "id": "0d871abd0017f444afc10c4e5c238f8d"
      }
    ]
  }
```


```
Number of unique event types in the corpus (across all splits): 168
```

Analysis 2: Pre-variable-level Overview

We describe the corpus before defining model-ready variables: document-level lengths (sentences, tokens), token-count statistics (vocabulary size, hapax count), and annotation counts (events and negative triggers per document). This characterises the raw text and annotations that will later be turned into features.

Observations:

- We characterised the MAVEN corpus at a pre-variable level by computing, for each document in the train and validation splits, the number of sentences, total tokens, in-document vocabulary size, number of hapax legomena, number of events, and number of negative triggers.
- Summary statistics and side-by-side comparison showed that train and validation have similar distributions for these quantities, with no missing values in the analysed fields.
- Document length (sentences and tokens) and annotation counts (events and negative triggers) are right-skewed, with a majority of short to medium documents and a long tail of longer, more event-dense documents.

In [4]:
```
def prevariable_stats(df):
    """
    Computes per-document statistics from a MAVEN event detection dataset

    Parameters
    ----------
    df : pd.DataFrame
        Dataframe in MAVEN format: each row is a document with at least a
        (list of sentence dicts with "tokens"), and optionally columns 'e
        'negative_triggers'.

    Returns
    -------
    pd.DataFrame
        Dataframe with one row per document containing the following colu
            - n_sentences: Number of sentences in document
            - n_tokens: Number of tokens in document
            - vocab_size: Number of unique tokens (vocabulary size) in do
            - n_hapax: Number of tokens that occur only once (hapax legom
            - n_events: Number of events in document (if present)
            - n_negative_triggers: Number of negative triggers in documen
    """
    # get the num of sentences, tokens, events, and negative triggers
    n_sentences = df["content"].map(len)
    n_tokens = df["content"].map(lambda c: sum(len(s["tokens"]) for s in
    n_events = df["events"].map(len) if "events" in df.columns else None
    n_neg = df["negative_triggers"].map(len) if "negative_triggers" in df

    # For each document, extract tokens and compute vocabulary size and h
    def vocab_and_hapax(content_list):
        tokens = [t for sent in content_list for t in sent["tokens"]]
        fdist = FreqDist(tokens)
        return len(fdist), len(fdist.hapaxes())

    # Apply the function to each document in the dataframe
    vocab_size, n_hapax = zip(*df["content"].map(vocab_and_hapax))

    # Create a dataframe with the computed statistics
    out = pd.DataFrame({"n_sentences": n_sentences, "n_tokens": n_tokens,
    if n_events is not None:
        out["n_events"] = n_events
    if n_neg is not None:
        out["n_negative_triggers"] = n_neg
    return out

# Compute per-document stats
stats_train = prevariable_stats(df_train)
stats_val = prevariable_stats(df_val)

# One combined dataframe with split and doc_id
stats_train_labeled = stats_train.assign(split="train")
stats_val_labeled = stats_val.assign(split="valid")
stats_all = pd.concat([stats_train_labeled, stats_val_labeled], ignore_in
stats_all.insert(0, "doc_id", np.concatenate([df_train["id"].values, df_v

# Summary dataframe: train vs valid side by side (mean, std)
summary_side_by_side = pd.DataFrame({
    "train_mean": stats_train.mean(),
    "train_std": stats_train.std(),
```

```
    "valid_mean": stats_val.mean(),
    "valid_std": stats_val.std(),
})

# Print all the stats
print("\033[1;33m\nCombined per-document stats (first 15 rows):\033[0m");
print("\033[1;33m\nSummary (train vs valid):\033[0m"); display(summary_si
print("\033[1;33m\nDescribe by split:\033[0m"); display(stats_all.groupby


# Define the columns to plot histograms
cols = ["n_sentences", "n_tokens", "n_events", "n_negative_triggers", "vo

# Plot for train
fig, axes = plt.subplots(2, 3, figsize=(12, 8))
for ax, col in zip(axes.flat, cols):
    if col in stats_train.columns:
        sns.histplot(stats_train[col], kde=True, ax=ax, bins=30)
        ax.set_title(col)
    else:
        ax.axis('off')
plt.suptitle("Pre-variable distributions (train)", fontsize=12)
plt.tight_layout()
plt.show()

# Plot for validation
fig, axes = plt.subplots(2, 3, figsize=(12, 8))
for ax, col in zip(axes.flat, cols):
    if col in stats_val.columns:
        sns.histplot(stats_val[col], kde=True, ax=ax, bins=30, color='ora
        ax.set_title(col)
    else:
        ax.axis('off')
plt.suptitle("Pre-variable distributions (validation)", fontsize=12)
plt.tight_layout()
plt.show()
```

```
Combined per-document stats (first 15 rows):
```

	doc_id	n_sentences	n_tokens	vocab_size	n_hap
0	8307a6b61b84d4eea42c1dd5e6e2cdba	9	352	175	1
1	387fe1dfe55067eb29e1fd4116d37af3	18	463	234	1
2	268c4763208c87ed7ebf55565c274d23	11	303	166	1
3	c95e68565081126b5c949117e423695a	10	309	162	1
4	3bec0b60c0940c5e46ee2cfc9504df92	10	213	133	9
5	39c2db9e18cd4a02b9aa8c1a3c58aab7	13	267	145	1
6	c1ae08941a115c7d45e205724e5aa3be	39	1215	472	3
7	ef451445d348d837669fbab71d1db3a2	12	295	180	1
8	045be42474c6c6225401d4b8cc03fe5e	8	195	105	1
9	eeb1b3357c409a5f55f45158ce2a62e5	7	118	84	6
10	5c2b32634deb09bf1c67e846cef14e44	9	245	112	1
11	f5f617fe0cbadf8fdbc06b0ecbe0be92	7	139	91	6
12	59d35dfd8d9fe91d66d11497d051aba5	30	824	402	2
13	ab70a3e49966caa8c35f8b27fabea3ad	15	379	216	1
14	ed34034eebbb1fa5144e33bdc12b5936	22	503	261	1

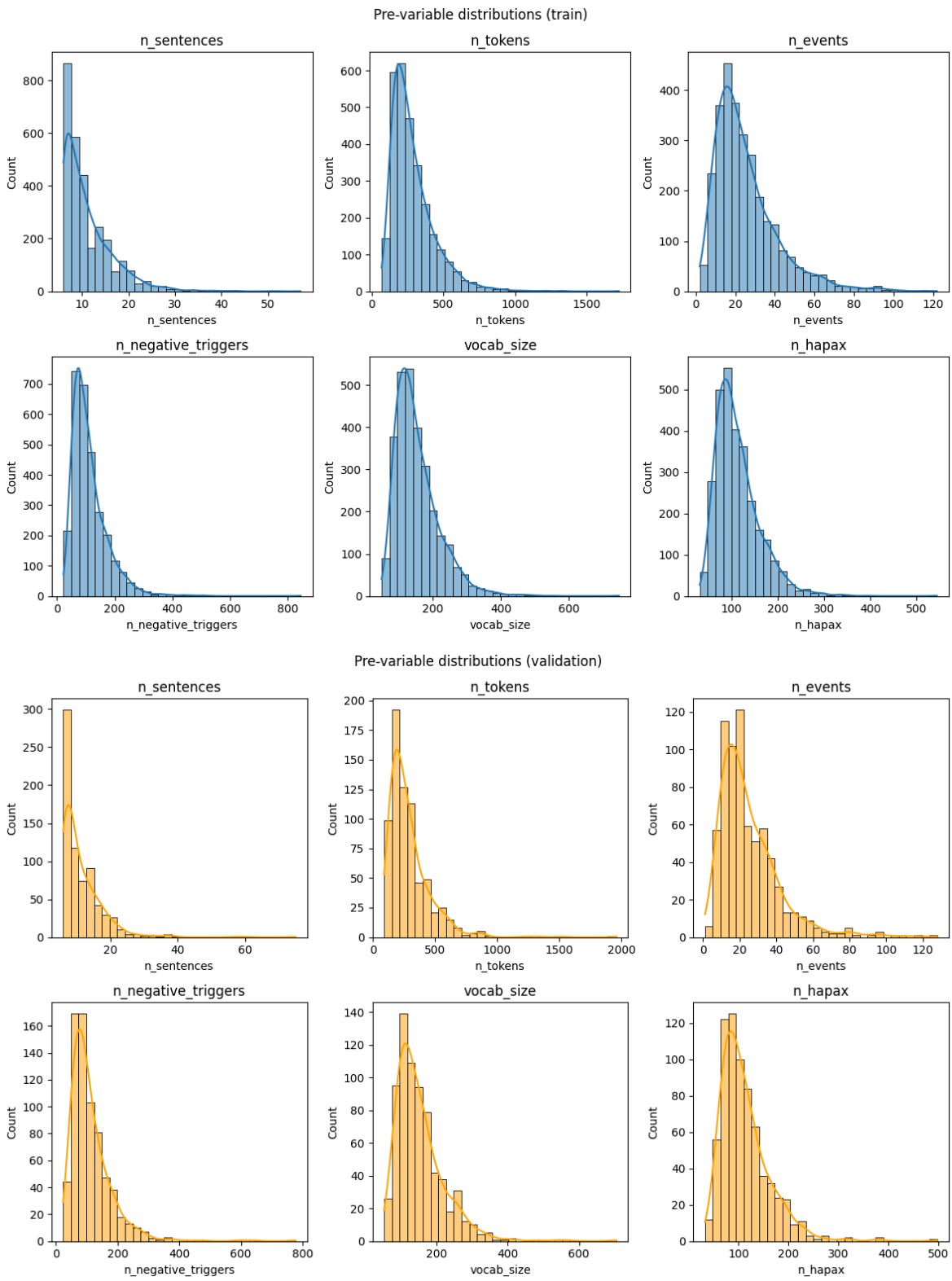
Summary (train vs valid):

	train_mean	train_std	valid_mean	valid_std
n_sentences	11.133196	5.468577	11.326761	6.460926
n_tokens	285.680055	152.267703	288.107042	172.912803
vocab_size	152.967044	64.632639	153.097183	69.264933
n_hapax	112.732578	47.421373	112.452113	49.685639
n_events	25.230347	16.089320	24.966197	16.614232
n_negative_triggers	111.222794	61.776284	112.252113	69.998709

Describe by split:

	n_sentences								n_tokens	
	count	**mean**	**std**	**min**	**25%**	**50%**	**75%**	**max**	**count**	**mean**
split										
train	2913.0	11.133196	5.468577	6.0	7.0	10.0	13.0	57.0	2913.0	285.680055
valid	710.0	11.326761	6.460926	6.0	7.0	9.0	14.0	75.0	710.0	288.107042

2 rows × 48 columns



Analysis 3: Univariate-level Overview

We look at one variable at a time: event type (and type_id) distribution across the corpus, trigger-word frequency, and the class balance between event triggers and negative triggers. This yields priors for event-type classification and for binary trigger detection, and shows the long-tail nature of the event ontology.

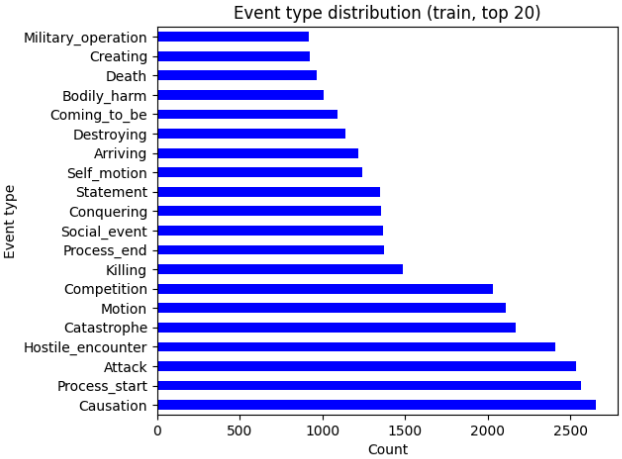
In [5]:
```
def get_event_type_series(df):
    """Collect (type, type_id) for every event in df. Returns (type_serie
    all_types = []
```

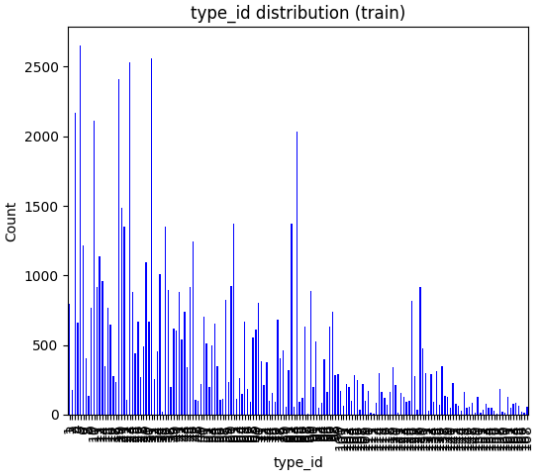
```
    all_type_ids = []
    for _, row in df.iterrows():
        for ev in row["events"]:
            all_types.append(ev["type"])
            all_type_ids.append(ev["type_id"])
    return pd.Series(all_types), pd.Series(all_type_ids)

def get_trigger_balance(df):
    """Return (n_positive, n_negative, total) trigger spans."""
    n_positive = sum(len(ev.get("mention", [])) for _, row in df.iterrows
    n_negative = df["negative_triggers"].map(len).sum()
    total = n_positive + n_negative
    return n_positive, n_negative, total

def get_trigger_word_counts(df):
    """Return Series of trigger-word value counts."""
    words = []
    for _, row in df.iterrows():
        for ev in row["events"]:
            for m in ev.get("mention", []):
                w = m.get("trigger_word")
                if w is not None:
                    words.append(w)
    return pd.Series(words).value_counts()

def run_univariate_for_split(df, split_name, n_top=20, plot=True):
    """Run full univariate analysis for one split (train or valid)."""
    type_series, type_id_series = get_event_type_series(df)
    n_events = len(type_series)

    print(f"\033[1;33m\n{split_name.upper()}\033[0m")
    print(f"\033[1mEvent type (categorical) — value counts (top {n_top}):
    print(type_series.value_counts().head(n_top))
    print("\033[1m\nEvent type_id (numeric) — describe:\033[0m")
    print(type_id_series.describe())
    prior_top = type_series.value_counts().head(n_top) / n_events
    print(f"\033[1m\nPrior P(type) — top {n_top}:\033[0m")
    print(prior_top)

    n_pos, n_neg, total = get_trigger_balance(df)
    print(f"\033[1m\nTrigger vs negative:\033[0m \n - positive: {n_pos},
    print(f"\033[1mPriors: \033[0m \n - P(positive) = {n_pos / total:.4f}
    print("\033[1m\nTop 20 trigger words:\033[0m")
    print(get_trigger_word_counts(df).head(20))

    if plot:
        fig, axes = plt.subplots(1, 2, figsize=(12, 5))
        # Set color: train = blue, val = orange
        color = "blue" if split_name.lower() == "train" else "orange"
        type_series.value_counts().head(n_top).plot(kind="barh", ax=axes[
        axes[0].set_xlabel("Count")
        axes[0].set_ylabel("Event type")
        axes[0].set_title(f"Event type distribution ({split_name}, top {n
        type_id_series.value_counts().sort_index().plot(kind="bar", ax=ax
        axes[1].set_xlabel("type_id")
        axes[1].set_ylabel("Count")
        axes[1].set_title(f"type_id distribution ({split_name})")
        plt.tight_layout()
        plt.show()
```

```python
# Run for both splits
for name, data in [("train", df_train), ("valid", df_val)]:
    run_univariate_for_split(data, name, n_top=20, plot=True)
```

```
TRAIN
Event type (categorical) — value counts (top 20):
Causation             2653
Process_start         2562
Attack                2535
Hostile_encounter     2411
Catastrophe           2172
Motion                2109
Competition           2032
Killing               1487
Process_end           1374
Social_event          1369
Conquering            1353
Statement             1350
Self_motion           1241
Arriving              1217
Destroying            1136
Coming_to_be          1093
Bodily_harm           1007
Death                  963
Creating               924
Military_operation     920
Name: count, dtype: int64

Event type_id (numeric) — describe:
count    73496.000000
mean        51.593787
std         40.285666
min          1.000000
25%         19.000000
50%         40.000000
75%         82.000000
max        168.000000
dtype: float64

Prior P(type) — top 20:
Causation             0.036097
Process_start         0.034859
Attack                0.034492
Hostile_encounter     0.032805
Catastrophe           0.029553
Motion                0.028695
Competition           0.027648
Killing               0.020232
Process_end           0.018695
Social_event          0.018627
Conquering            0.018409
Statement             0.018368
Self_motion           0.016885
Arriving              0.016559
Destroying            0.015457
Coming_to_be          0.014872
Bodily_harm           0.013701
Death                 0.013103
Creating              0.012572
Military_operation    0.012518
Name: count, dtype: float64

Trigger vs negative:
```

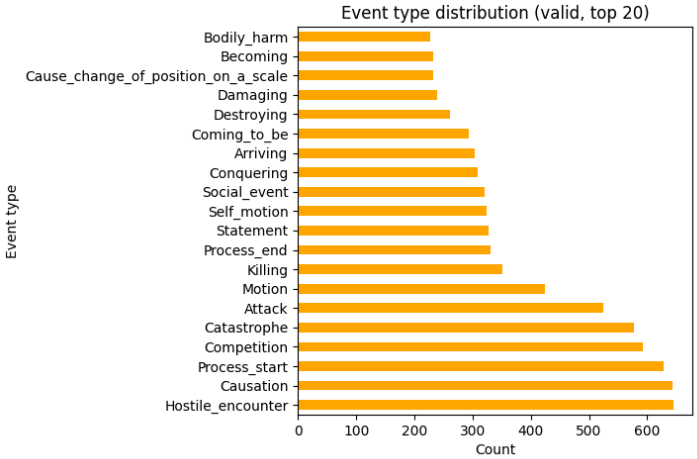
```
 - positive: 77993,
 - negative: 323992,
 - total: 401985
Priors:
 - P(positive) = 0.1940,
 - P(negative) = 0.8060

Top 20 trigger words:
killed        828
storm         775
began         770
held          649
hurricane     623
caused        607
attack        586
took place    586
occurred      536
damage        509
became        499
battle        450
tour          439
fought        389
war           368
tournament    346
Battle        330
destroyed     330
ended         316
captured      312
Name: count, dtype: int64
```

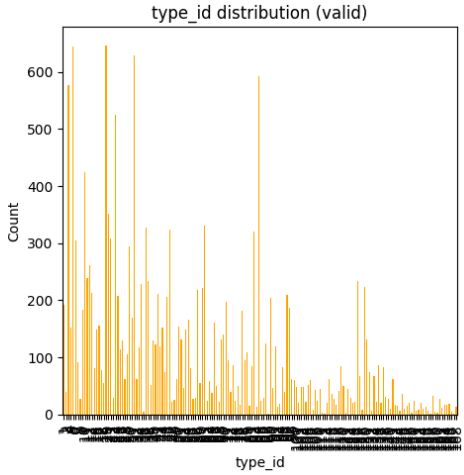




```
VALID
Event type (categorical) — value counts (top 20):
Hostile_encounter                      646
Causation                              644
Process_start                          629
Competition                            593
Catastrophe                            577
Attack                                 524
Motion                                 424
Killing                                352
Process_end                            331
Statement                              327
Self_motion                            324
Social_event                           320
Conquering                             308
Arriving                               304
Coming_to_be                           294
Destroying                             261
Damaging                               239
Cause_change_of_position_on_a_scale    233
Becoming                               233
Bodily_harm                            228
Name: count, dtype: int64

Event type_id (numeric) — describe:
count    17726.000000
mean        52.191809
std         40.382834
min          1.000000
25%         19.000000
50%         41.000000
75%         82.000000
max        168.000000
dtype: float64

Prior P(type) — top 20:
Hostile_encounter                      0.036444
Causation                              0.036331
Process_start                          0.035485
Competition                            0.033454
Catastrophe                            0.032551
Attack                                 0.029561
Motion                                 0.023920
Killing                                0.019858
Process_end                            0.018673
Statement                              0.018447
Self_motion                            0.018278
Social_event                           0.018053
Conquering                             0.017376
Arriving                               0.017150
Coming_to_be                           0.016586
Destroying                             0.014724
Damaging                               0.013483
Cause_change_of_position_on_a_scale    0.013145
Becoming                               0.013145
Bodily_harm                            0.012862
Name: count, dtype: float64

Trigger vs negative:
```

```
 - positive: 18904,
 - negative: 79699,
 - total: 98603
Priors:
 - P(positive) = 0.1917,
 - P(negative) = 0.8083

Top 20 trigger words:
killed        176
storm         172
began         163
held          157
hurricane     155
took place    146
became        141
caused        138
occurred      130
attack        117
damage        113
Battle        110
battle        109
destroyed     100
fought         91
war            90
tournament     89
reported       87
played         83
conflict       77
Name: count, dtype: int64
```





Analysis 4: Baseline

We derive trivial baseline scores from the corpus priors (trigger vs negative and event-type distribution) so we have a lower bound any real model must beat. We then briefly reference the MAVEN benchmark (Wang et al., 2020): BERT+CRF reaches 67.8 F1 on MAVEN trigger classification; we use this model as our event extractor for the Tell Me Again pipeline and do not aim to improve MAVEN event detection itself.

In [6]:
```
# Calculate priors for trigger vs negative trigger detection
n_pos, n_neg, total = get_trigger_balance(df_train)
p_neg = n_neg / total
p_pos = n_pos / total
```

```
# Calculate priors for event type classification
type_series, _ = get_event_type_series(df_train)
n_events = len(type_series)
majority_type = type_series.value_counts().index[0]
majority_count = type_series.value_counts().iloc[0]
p_majority_type = majority_count / n_events

print(f"\033[1;33m\nPrior baselines (train)\033[0m")
print(f"\033[1m  • Trigger vs negative:\033[0m \n - P(negative) = {p_neg:
print(f"\033[1m  • Event type:\033[0m \n - majority type = '{majority_typ

# Print a summary table of the baseline accuracies
print("\033[1;33m\nBaseline accuracies from corpus priors (train set)\033
baseline_summary = pd.DataFrame({
    "Baseline": [
        "Always negative (trigger detection)",
        "Majority event type (type classification)",
    ],
    "Train accuracy": [p_neg, p_majority_type],
})
display(baseline_summary)

# 2) MAVEN paper benchmark (for methodology / related work)
print("\nMAVEN benchmark (Wang et al., 2020, Table 5 — trigger classifica
print("  • BERT+CRF: P=65.0, R=70.9, F1=67.8 (best in paper)")
print("  • We use BERT+CRF as the event extractor; our thesis focus is on
```

```
Prior baselines (train)
  • Trigger vs negative:
 - P(negative) = 0.8060 → 'always predict negative' accuracy = 0.8060
  • Event type:
 - majority type = 'Causation' (P = 0.0361) → 'always predict majority' ac
curacy = 0.0361

Baseline accuracies from corpus priors (train set)
```

	Baseline	Train accuracy
0	Always negative (trigger detection)	0.805980
1	Majority event type (type classification)	0.036097

```
MAVEN benchmark (Wang et al., 2020, Table 5 — trigger classification on MA
VEN test):
  • BERT+CRF: P=65.0, R=70.9, F1=67.8 (best in paper)
  • We use BERT+CRF as the event extractor; our thesis focus is on Tell Me
Again, not improving MAVEN ED.
```

Dataset 2: TellMeAgain!

Data Loading

In [7]:
```
# Define the core directories
PROJECT_ROOT = os.path.abspath(os.path.join(os.path.dirname("."), ".."))
TELL_ME_AGAIN_ZIP_RAW = os.path.join(PROJECT_ROOT, "data", "raw", "TellMe

# Load the story-level corpus via zip file
```

```
storyDataset = StoryDataset(data_path=TELL_ME_AGAIN_ZIP_RAW)
similarityDataset = SimilarityDataset(data_path=TELL_ME_AGAIN_ZIP_RAW)
```

```
Loading summaries: 100%|██████████| 29505/29505 [00:19<00:00, 1501.93it/s]
Loading summaries: 100%|██████████| 29505/29505 [00:19<00:00, 1495.87it/s]
```

Analysis 1: Corpus-level Overview

In the cell below:

- We iterate over each story in the TellMeAgain dataset.
- For each story, we collect various statistics, including the number of summaries in original and deduplicated form, presence of English or anonymized summaries, sentence count statistics, number of genres, and available languages.
- Each row in the resulting DataFrame represents a single story, containing all the collected statistics (e.g., Wikidata ID, number of summaries, languages, sentence counts, genres) for that story, allowing for easy analysis and display of the entire corpus.

Observations:

- Number of sentences with respect to languages differ: Although EN, DE, and FR have similar lengths/stds, IT and ES have significantly lower length.
- Singletons: Stories for which only one summary was collected in the original language, so there is only a single summary in the cluster.
- Deduplication: If the translated summaries of the same work has > 0.6 cosine similarity, the one with lower language-priority is filtered out at API runtime (`get_all_summaries_en()`)
- English summaries are not deduplicated as they are always the original summaries, not translations, so the 'n_deduped' will be 0 for them.
- 423 stories did have multiple summaries, however the deduplication process removed all of these summaries, making them singletons.

In [8]:
```
# Build a statistics dataframe for the corpus where each row is a story a
records = []
for story in storyDataset:

    # Raw cluster size (all original languages)
    numSummariesRaw = len(story.summaries_original)
    languagesRaw = list(story.summaries_original.keys())

    # Get the number of deduplicated summaries (i.e., summaries that beco
    ids_deduped, summaries_deduped = story.get_all_summaries_en()
    numSummariesDeduped = len(summaries_deduped)

    # Get the number of anonymized summaries
    numSummariesAnonymized = len(story.summaries_anonymized) if story.sum

    # Get the sentence count statistics across all language versions of t
    numSentences = list(story.num_sentences.values())
    avgSentences = sum(numSentences) / len(numSentences) if numSentences
    maxSentences = max(numSentences) if numSentences else 0
```

```
    minSentences = min(numSentences) if numSentences else 0
    stdSentences = np.std(numSentences) if numSentences else 0

    # Append all of the statistics to the records
    records.append({
        "wikidata_id":         story.wikidata_id,                        #
        "n_summaries_raw":     numSummariesRaw,                          #
        "n_summaries_deduped": numSummariesDeduped,                      #
        "has_en":              "en" in story.summaries_original,         #
        "n_anon_langs":        numSummariesAnonymized,                   #
        "has_anonymized":      numSummariesAnonymized > 0,               #
        "has_translated_summaries": bool(story.summaries_translated),    #
        "avg_sentences":       avgSentences,                             #
        "max_sentences":       maxSentences,                             #
        "min_sentences":       minSentences,                             #
        "std_sentences":       stdSentences,                             #
        "n_genres":            len(story.genres),                        #
        "langs":               languagesRaw,                             #
    })

# Create a dataframe from the records dictionary
story_df = pd.DataFrame(records)

# Print the dataframe
story_df.head()

# Print the dataframe shape
print("\033[1;33m\nDataframe shape:\033[0m")
print(f"  {story_df.shape[0]:,} rows × {story_df.shape[1]} columns")

# Get the corpus dimensions
total_stories    = len(story_df)
total_raw        = story_df["n_summaries_raw"].sum()
total_deduped    = story_df["n_summaries_deduped"].sum()
singleton_count_before_dedup = (story_df["n_summaries_raw"] == 1).sum()
singleton_count_after_dedup  = (story_df["n_summaries_deduped"] == 1).sum

# Print the corpus dimensions
print("\033[1;33m\nCorpus dimensions:\033[0m")
print(f"  - Total stories (unique works):          {total_stories:>40,}")
print(f"  - Total summaries - raw (all languages): {total_raw:>40,}")
print(f"  - Total summaries - after deduplication: {total_deduped:>40,}")
print(f"  - Stories with translated summaries:     {story_df['has_transla
print(f"  - Stories with an English summary:       {story_df['has_en'].su
print(f"  - Stories with anonymized summaries:     {story_df['has_anonymi
print(f"  - Singleton clusters (n_deduped == 1):   {singleton_count_after
print(f"  - Singleton clusters (number of summaries before deduplication

# Create a table of the summary count by language
lang_stats = storyDataset.get_lang_stats()
lang_df = (
    pd.DataFrame({
        "language":       list(lang_stats["languages"].keys()),
        "n_raw":          list(lang_stats["languages"].values()),
        "n_deduped":      [lang_stats["languages_direct_translations_remo
                           for k in lang_stats["languages"].keys()],
    })
    .sort_values("n_raw", ascending=False)
    .reset_index(drop=True)
)
```

```
# Print the number of summaries per language
print("\033[1;33m\nNumber of summaries per language:\033[0m")
print("(Note: English summaries are not deduplicated as they are always t
display(lang_df)

# Create a table of the summary length (sentences) per language version
print("\033[1;33m\nNumber of sentences per language version:\033[0m")
sent_records = []
for story in storyDataset:
    for lang, n in story.num_sentences.items():
        sent_records.append({"lang": lang, "n_sentences": n})
sent_df = pd.DataFrame(sent_records)
display(sent_df.groupby("lang")["n_sentences"].describe())

# Check missing values
no_anon = sum(1 for s in storyDataset if not s.summaries_anonymized)
no_genre = sum(1 for s in storyDataset if len(s.genres) == 0)
no_en = sum(1 for s in storyDataset if "en" not in s.summaries_original)
no_sim = sum(1 for s in storyDataset if len(s.similarities) == 0)
no_summaries_original = sum(1 for s in storyDataset if not s.summaries_or
no_summaries_translated = sum(1 for s in storyDataset if not s.summaries_
no_titles             = sum(1 for s in storyDataset if not s.titles)
no_num_sentences      = sum(1 for s in storyDataset if not s.num_sentence
no_sentences          = sum(1 for s in storyDataset if not s.sentences)
empty_sim_labels      = sum(1 for s in storyDataset if not (getattr(s, "s
no_description        = sum(1 for s in storyDataset if not (s.description
def _title_string(s):
    """Return a usable title string for a story, using 'title' or entries
    t = getattr(s, "title", None)
    if isinstance(t, str) and t.strip():
        return t.strip()
    if isinstance(t, dict) and t.get("value"):
        return str(t["value"]).strip()
    titles = getattr(s, "titles", None) or {}
    for v in (titles or {}).values():
        if v and str(v).strip():
            return str(v).strip()
    return ""
no_usable_title = sum(1 for s in storyDataset if not _title_string(s))


# Print the missing values
print("\033[1;33m\nMissing values:\033[0m")
print(f"  - Stories with no anonymized summaries: {no_anon}")
print(f"  - Stories with no genres: {no_genre}")
print(f"  - Stories with no English original (but can have multiple EN su
print(f"  - Stories with no similarities matrix: {no_sim}")
print(f"  - Stories with empty summaries_original:   {no_summaries_origin
print(f"  - Stories with empty summaries_translated: {no_summaries_transl
print(f"  - Stories with empty titles dict:          {no_titles}")
print(f"  - Stories with empty num_sentences:       {no_num_sentences}")
print(f"  - Stories with empty sentences:            {no_sentences}")
print(f"  - Stories with empty similarities_labels: {empty_sim_labels}")
print(f"  - Stories with blank description:        {no_description}")
print(f"  - Stories with blank title:               {no_usable_title}")


# Plots
fig, axes = plt.subplots(2, 3, figsize=(16, 10))
```

```
# Raw cluster size distribution
cs_raw = story_df["n_summaries_raw"].value_counts().sort_index()
cs_raw.plot(kind="bar", ax=axes[0, 0], color="steelblue")
axes[0, 0].set_title("Cluster size — raw (all languages)")
axes[0, 0].set_xlabel("# summaries per story"); axes[0, 0].set_ylabel("#

# Deduplicated cluster size distribution (what retrieval eval sees)
cs_ded = story_df["n_summaries_deduped"].value_counts().sort_index()
cs_ded.plot(kind="bar", ax=axes[0, 1], color="darkorange")
axes[0, 1].set_title("Cluster size — after deduplication")
axes[0, 1].set_xlabel("# summaries per story (deduped)"); axes[0, 1].set_

# Language distribution
lang_df.plot(x="language", y=["n_raw", "n_deduped"], kind="bar", ax=axes[
axes[0, 2].set_title("Summary count by language")
axes[0, 2].set_xlabel("Language"); axes[0, 2].set_ylabel("# summaries")
axes[0, 2].legend(["raw", "after dedup"])

# Sentence-count distribution per summary (EN only, then all)
sns.histplot(sent_df[sent_df["lang"] == "en"]["n_sentences"],
             kde=True, bins=50, ax=axes[1, 0], color="steelblue",
             label="en", stat="density")
sns.histplot(sent_df[sent_df["lang"] != "en"]["n_sentences"],
             kde=True, bins=50, ax=axes[1, 0], color="darkorange",
             alpha=0.6, label="non-en", stat="density")
axes[1, 0].set_title("Sentence count per summary (EN vs non-EN)")
axes[1, 0].set_xlabel("# sentences"); axes[1, 0].legend()

# Anonymized language coverage
anon_dist = story_df["n_anon_langs"].value_counts().sort_index()
anon_dist.plot(kind="bar", ax=axes[1, 1], color="mediumseagreen")
axes[1, 1].set_title("# anonymized versions per story")
axes[1, 1].set_xlabel("# anonymized languages"); axes[1, 1].set_ylabel("#

# Genre count per story
story_df["n_genres"].value_counts().sort_index().plot(kind="bar", ax=axes
axes[1, 2].set_title("Wikidata genre count per story")
axes[1, 2].set_xlabel("# genres"); axes[1, 2].set_ylabel("# stories")

plt.suptitle("Analysis 1: Corpus-Level Overview — TellMeAgain!", fontsize
plt.tight_layout()
plt.show()

# One full example Story
ex = list(storyDataset)[0]
ids_ex, sums_ex = ex.get_all_summaries_en()
print(f"\033[1;33m\nExample Story:\033[0m \n{ex}")
print(f"  wikidata_id:   {ex.wikidata_id}")
print(f"  title:         {ex.title}")
print(f"  description:   {ex.description}")
print(f"  Languages present: {list(ex.summaries_original.keys())}")
print(f"  num_sentences:     {ex.num_sentences}")
print(f"  genres (Q-IDs):    {ex.genres}")
print(f"  anonymized langs:  {list(ex.summaries_anonymized.keys()) if ex.
print(f"  get_all_summaries_en → {len(ids_ex)} summaries after dedup")
print(f"\n  EN summary (first 400 chars):\n  {ex.summaries_original.get('

# Print all example summaries
print("\n\033[1;33mExample summaries:\033[0m")
```

```
for i, summary in enumerate(sums_ex):
    print(f"\n  Summary {i+1}:")
    print(f"    {summary}")
```

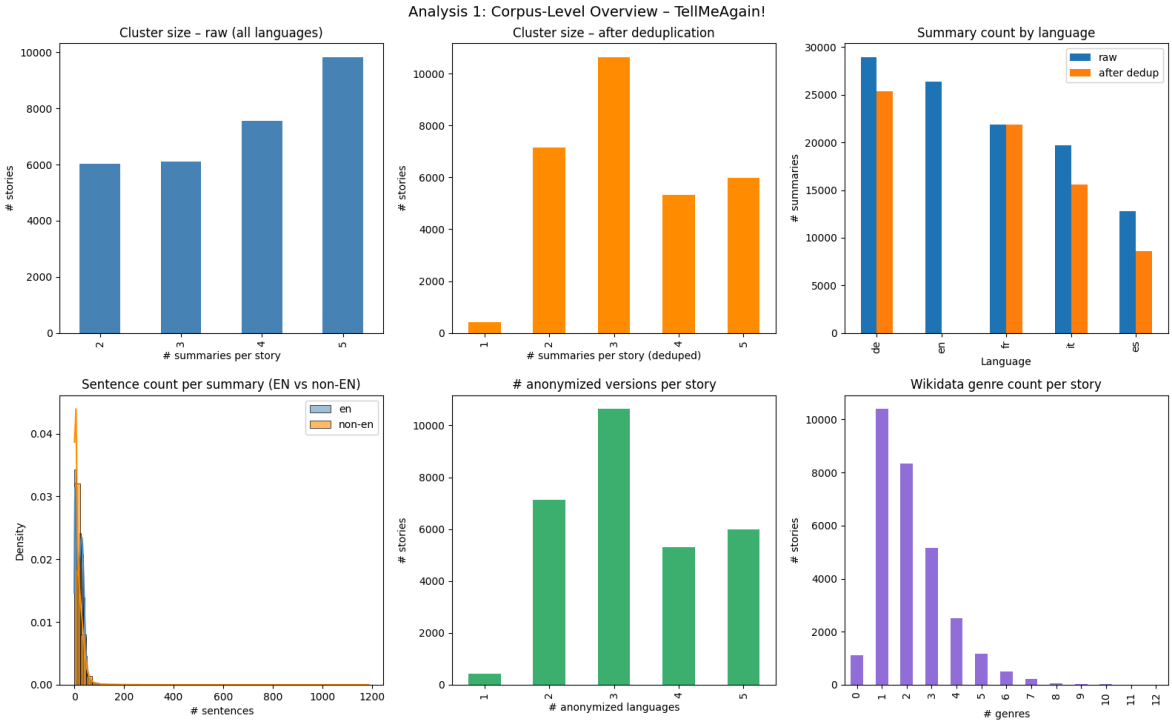
```
Dataframe shape:
  29,505 rows × 13 columns

Corpus dimensions:
  - Total stories (unique works):
29,505
  - Total summaries — raw (all languages):
109,691
  - Total summaries — after deduplication:
97,813
  - Stories with translated summaries:
29,504  (100.0%)
  - Stories with an English summary:
26,385  (89.4%)
  - Stories with anonymized summaries:
29,505  (100.0%)
  - Singleton clusters (n_deduped == 1):
423  (1.4%) --> should be excluded from retrieval eval
  - Singleton clusters (number of summaries before deduplication == 1):
0  (0.0%)  --> should be excluded from retrieval eval
100%|██████████| 29505/29505 [00:00<00:00, 95514.84it/s]

Number of summaries per language:
(Note: English summaries are not deduplicated as they are always the origi
nal summaries, not translations, so the 'n_deduped' will be 0)
```

	language	n_raw	n_deduped
0	de	28947	25408
1	en	26385	0
2	fr	21862	21848
3	it	19737	15548
4	es	12760	8624

```
Number of sentences per language version:
```

	count	mean	std	min	25%	50%	75%	max
lang								
de	28942.0	22.877721	20.513467	1.0	10.0	18.0	30.0	517.0
en	26385.0	22.517567	16.947168	0.0	8.0	22.0	33.0	261.0
es	12754.0	15.606241	21.156852	1.0	4.0	7.0	24.0	1188.0
fr	21848.0	11.004760	16.807870	1.0	3.0	5.0	11.0	418.0
it	19733.0	13.733746	14.424066	1.0	4.0	9.0	20.0	270.0

```
Missing values:
  - Stories with no anonymized summaries: 0
  - Stories with no genres: 1112
  - Stories with no English original (but can have multiple EN summaries):
3120
  - Stories with no similarities matrix: 0
  - Stories with empty summaries_original:   0
  - Stories with empty summaries_translated: 1
  - Stories with empty titles dict:          0
  - Stories with empty num_sentences:       0
  - Stories with empty sentences:            0
  - Stories with empty similarities_labels: 0
  - Stories with blank description:        32
  - Stories with blank title:                0
```



```
Example Story:
<Story title='Reckless' description='2010 young adult novel by Cornelia Fu
nke'>
  wikidata_id:   Q1000352
  title:         Reckless
  description:   2010 young adult novel by Cornelia Funke
  Languages present: ['en', 'de', 'it']
  num_sentences:     {'de': 28, 'it': 6, 'en': 63}
  genres (Q-IDs):    ['Q132311', 'Q111984153']
  anonymized langs:  ['en', 'de', 'it']
  get_all_summaries_en → 3 summaries after dedup

  EN summary (first 400 chars):
  Reckless opens with twelve-year-old Jacob Reckless using a mirror to ent
er the magical world of Mirrorworld.
Twelve years later, Jacob's brother Will follows Jacob into the Mirrorworl
d. Will is attacked by a Goyl, a humanoid race with stone skin. As a resul
t of this, Will's skin begins turning to stone. Jacob knows that the stone
will soon invade Will's entire body, and Will will become one of the...

Example summaries:

  Summary 1:
    Reckless opens with twelve-year-old Jacob Reckless using a mirror to e
nter the magical world of Mirrorworld.
Twelve years later, Jacob's brother Will follows Jacob into the Mirrorworl
d. Will is attacked by a Goyl, a humanoid race with stone skin. As a resul
t of this, Will's skin begins turning to stone. Jacob knows that the stone
will soon invade Will's entire body, and Will will become one of the Goyl
and he rides off in search of a cure. He is told that the berries that gro
w in the garden of child-eating witch may cure Will of the curse. Will sne
aks back through the mirror to say goodbye to his girlfriend Clara over th
e phone, leading to her going to the Reckless brothers' house and entering
the mirror.
Jacob, Will, Clara and Jacob's vixen friend, Fox, journey to the witch's h
ouse, which lies deserted. When they arrive, they realize that they're bei
ng pursued by a dangerous creature called The Tailor. Jacob and Fox fight
The Tailor, with Will and Clara inside the gates, which The Tailor cannot
enter, but Jacob is wounded on the shoulder during the fight. Will takes t
he berries, and they sleep. In the morning, Will's condition has not chang
ed; the berries did not work. In another attempt to save his brother, Jaco
b decides to visit the Red Fairy, in the hope that he can convince her to
help break the curse. Jacob forces a dwarf named Valiant to help the group
find her. Meanwhile, the Goyl King's right hand leads a group of Goyl sold
iers to find Will, because they believe he is destined to become the ultim
ate protector of their king, as his skin is turning to jade.
The Goyl patrol attacks the group before they get to the Red Fairy, but Va
liant is able to show them how to enter the secret valley where she is. Ja
cob then rows to the Fairies' Island while the rest of the party wait on t
he shore of the lake. Miranda, the Red Fairy, tells Jacob that the curse w
ill be lifted if the Dark Fairy, who created the curse, was destroyed. Jac
ob puts Will into a deep sleep to stop the spread of the stone till the cu
rse can be lifted by giving him an enchanted rose that Will pricks his fin
ger on. Valiant betrays the group and tells the Goyl soldiers how to enter
the secret valley. They attack the group, killing Jacob and taking the sle
eping Will captive. Valiant, who has developed a soft spot for Clara, stay
s behind, and begins to dig a grave for Jacob. While he does this, the Red
Fairy brings Jacob back from the dead.
The group travels to the Goyl city underneath the mountains to rescue Wil
```

l. Valiant makes a deal with Jacob to lead them there. An argument in the mountains leads to Jacob and Valiant deciding to leave Clara and Fox behind. Valiant manages to smuggle Jacob into the Goyl city. Inside the city, a merchant informs Valiant that Will is being held in the Floating Palace high above the mountains. Jacob climbs up to it after making himself invisible.
While Jacob is climbing the Palace Walls, he is attacked and captured, then questioned on the whereabouts of Clara and Fox. He is tortured and gives away their location, then passes into a deep sleep. When he wakes up, he finds himself in a cell next to a still sleeping Will, waiting to be woken by a kiss from his true love. The Dark Fairy arrives with Clara and Fox, who have been found and captured. Clara kisses Will, and he wakes. He is now completely Goyl, with no trace remaining of Will.
Will leaves his cell, so that he can protect the Goyl King, and Clara is jailed in the cell next to Jacob, with Fox locked up on the floor above them. Later, Valiant appears and rescues them all. They leave the city with a biplane.
On their way back to the tower where the Mirror stands, Jacob decides he will not give up on Will. He catches a train to the Empire's capital city, leaving the others to return to the tower on their own. Jacob understands that going there is his last chance to kill the Dark Fairy and undo the curse she put upon Will. The Goyl King is going to marry the Empress' daughter in order to make peace between the humans and the Goyl. Jacob tried to spy on the Dark Fairy and Will in the palace but Will hears him, breaks down the wall, and begins to fight with Jacob. The fight is interrupted and Jacob is given an audience with the Empress. Jacob tells her that he must kill the Dark Fairy because of his brother and needs her help. She sends one of her officials to tell Jacob that the Dark Fairy takes a walk thorough the Palace Gardens every evening, and offers to give him access. That evening, Jacob ambushes the Dark Fairy. He touches her and says her true name, which turns her into a tree. She then promises that she will turn Will back into his human form, if Jacob allows him to protect the King during the wedding to the Empress' daughter.
Jacob steals a golden ball from the Palace, which will trap anyone who catches it inside until it is polished, planning to use it on Will. Jacob attends the wedding, as do Clara, Valiant and Fox. The Empress has ordered an attack on the Goyl King and his people during the ceremony. The Cathedral turns into a bloodbath, and many of the humans die. Jacob realizes he has played into the Empress' hands and led the Goyl to a slaughter so he shouts her name to release her from the tree. The Black Moths of the Dark Fairy fill the Cathedral, poisoning hundreds of humans but Jacob protects Clara. The Goyl win the battle, and the wedding ceremony continues.
After the wedding, Jacob meets with the Dark Fairy, who tells him that she will return Will to his human form under the condition that Jacob take him very far away, because if she sees him again, she will kill him. Jacob manages to catch Will in the golden ball and the Dark Fairy returns him to his human form while still trapped.
Jacob then returns to the tower with Clara, Fox and Valiant. When they arrive at the tower, Jacob tells Clara to polish the golden ball until she sees her reflection. As she does this, Will is released, completely human again, and he embraces Clara. Will and Clara then leave MirrorWorld.
The book then sets up the start of the sequel, implying the price Jacob paid for the cure was a curse being put on him.

Summary 2:

New York: As a 12-year-old boy, Jacob, longing for his missing father, John Reckless, enters his room. There, he finds a message that takes him through a mirror into a world of mirrors. Where the characters of Grimm's Fairy Tales actually exist, but the world has evolved and modernized and is now in a state of decay.

From now on, he often visits the mirror world secretly, at fifteen he snea ks away for the first time for weeks. In his increasingly long periods of absence from his world, he becomes an apprentice to the treasure hunter Ch anute. Later, Jacob himself becomes a treasure hunter for the Empress of A ustria.
Twelve years after Jacob Reckless first entered the world behind the mirro r, his brother Will sees him disappear into the magic mirror and follows h im. There, Will is attacked by a goyl, which causes him to slowly grow a s kin of jade that transforms him into a goyl. Will's friend Clara follows h im through the mirror and now sets off with him and Jacob and his companio n Fox, who are trying to find an antidote to the curse. The first attempt to cure Will Reckless with berries from the stump of a child-eating witch from the Black Forest fails, Jacob and Fox narrowly escape the so-called "Schneider". The tailor makes clothes out of human skin.
At the same time, the Goyl under King Kamen and with the help of the Dark Fairy are waging war against the Imperial armies, and many people have alr eady fallen victim to the war. The Dark Fairy has a dream of a Jade Goyl, a legendary guardian in fairy tales, sent to save the Goyl in her darkest hour. And so the Goyl Hentzau goes with an armed troop in search of Will R eckless, the only one to grow the jade skin.
Meanwhile, Jacob and the others make their way to the Red Fairy Miranda, f rom whom he hopes to get help. But only the dwarf Evenaugh Valiant knows t he way there, and so he must trust him whether he likes it or not, even th ough the dwarf has betrayed him before. The Red Fairy promises to help Jac ob, but only on the condition that she destroy her sister, the Dark Fairy. When he returns from the fairy, his brother is kidnapped by Hentzau and so he must follow him underground with Valiant to free Will. But the attempt fails, and they must flee.
In order to restore peace, Empress Thérèse of Austria and King Kamen agree to his soon marriage to Thérèse's daughter. Will, who has now completely t ransformed into a goyl and forgotten his former life, is the king's bodygu ard. Shortly before the wedding, Jacob manages to capture the Dark Fairy b y pronouncing her name, which he learned from Miranda. Now he's trying to find Will and bring him to the Dark Fairy so she can turn him back.
The Empress learns that the Dark Fairy no longer protects Kamen and tries to have him and the Goyls present at the wedding, including Will, killed. In order to save his brother, Jacob is forced to free the Dark Fairy, who then appears and kills most of the people present. The Goyls will take the rest as hostages. Amazingly, the Dark Fairy turns Will back into a human, but threatens to kill him if Jacob doesn't take him far away. Will and Cla ra return to the other world, but Jacob Reckless is cursed by the Dark Fai ry when he pronounces her name and sets out to find an antidote, otherwise he will die within a year.

Summary 3:

Jacob Recless lives alone with his mother and younger brother Will aft er his father disappeared. One night, while everyone is asleep, driven by curiosity, he enters his father's study and discovers the existence of the Mirror of the Worlds, a portal to another dimension (called "The World Bey ond the Mirror"), populated by fantastic creatures, such as giants, fairie s, witches, Goyl, water genies, lorelei... But torn apart by a war between the realm of Empress Teresa of Austria and Kami'en, king of the Goyl. Frightened by reality, he often takes refuge in this fantasy world for 12 years, until, one night, Will also enters the mirror; due to a spell of th e Dark Fairy, he, wounded by a Goyl, begins to transform into a creature l ike the one who wounded him, with jade skin and golden eyes. Thus begins a race against time for Jacob, Fox (a creature friend of the boy) and Clara (Will's fiancée) to find the cure. But the shadow of the Fairy looms over them, with the sole purpose of capturing the Jade Goyl.

Analysis 2: Inspecting Stories with No Genres

We are exploring stories that are missing genre information and demonstrating how to identify them in the dataset in the next cell.

Results:

- If a stratified sampling will be done, these stories can be removed from the dataset.
- Also, if the train/test/val split is not representative of the full dataset with respect to Genre, these can be removed as well.

In [9]:
```
# Filter stories with no genres
stories_no_genre = [s for s in storyDataset if len(s.genres) == 0]
print(f"\nThere are {len(stories_no_genre)} stories with NO genres.")

if stories_no_genre:
    example_no_genre = stories_no_genre[0]
    print("\033[1;33m\nExample story with no genres:\033[0m")
    fields = [
        "wikidata_id", "title", "titles", "description",
        "summaries_original", "summaries_translated", "summaries_anonymiz
        "num_sentences", "sentences", "genres", "similarities_labels"
    ]
    for field in fields:
        print(f"\n--- {field} ---")
        value = getattr(example_no_genre, field, None)
        if isinstance(value, (dict, list)):
            import pprint
            pprint.pprint(value)
        else:
            print(value)

    print("\n--- similarities shape ---")
    similarities = getattr(example_no_genre, "similarities", None)
    if hasattr(similarities, "shape"):
        print(similarities.shape)
    else:
        print(type(similarities))
else:
    print("No stories without genres found.")
```

```
There are 1112 stories with NO genres.

Example story with no genres:

--- wikidata_id ---
Q1000933

--- title ---
Færeyinga Saga

--- titles ---
{'de': 'Färingersaga',
 'en': 'Færeyinga Saga',
 'es': 'Saga Færeyinga',
 'it': 'Saga dei Faroesi'}

--- description ---
literary work

--- summaries_original ---
{'de': 'Die Färingersaga beginnt mit den Worten (neufäröisch):\n'
       'Grímur Kamban var fyrsti maður ið búsettist í Føroyum.\n'
       'Hetta var á døgum Haralds Hárfagra,\n'
       'tá stór mannfjøld flýddi undan harðræði hansara.\n'
       'auf Deutsch:\n'
       'Grímur Kamban war der erste Mann, der sich auf den Färöern '
       'niederließ.\n'
       'Das war in den Tagen Harald Schönhaars,\n'
       'als viele Leute vor seiner Herrschsucht flohen.\n'
       'Es wird davon ausgegangen, dass jener Grímur tatsächlich gelebt ha
t '
       'und im 9. Jahrhundert von Norwegen auf die Färöer ging, sich in '
       'Funningur niederließ und so die nordische Landnahme auf dem Archip
el '
       'einleitete. Allerdings war das schon vor der Zeit Harald Schönhaar
s, '
       'wahrscheinlich bereits um 825. Offensichtlich verwechselt die '
       'Färingersaga diese erste Welle der Landnahme mit der zweiten Welle
um '
       'ca. 885–890, die tatsächlich eine Flucht vor Harald Schönhaars '
       'Herrschaft war.\n'
       'Auf den Färöern etablierte sich eine freie Republik von Auswandere
rn, '
       'und in ihrer Hauptstadt Tórshavn hatten sie ihr eigenes Ting. Dies
e '
       'Stätte namens Tinganes ist heute noch Regierungssitz. Die Färinger
saga '
       'enthält diese und weitere historische Fakten, vor allem die Geschi
chte '
       'des Sigmundur Brestisson (Sigmund), der die Färöer gegen den '
       'Widerstand des Trónd aus Gøta (Trond) um 1000 christianisierte. Tr
onds '
       'Rolle ist derart zentral, dass die Färingersaga auch „Die Sage von
'
       'Trond“ genannt wird.\n'
       'Da die Sage von Mönchen aufgeschrieben wurde, ist sie freilich nic
ht '
       'unparteiisch, sondern stellt Sigmund (den Christen) als den Guten
dar '
       'und Trond (den Heiden) als den Bösen. Es gibt darüber hinaus auch
```

```
'
        'Teile der Färingersaga, die ganz offensichtlich mehr Dichtung als
'
        'Wahrheit sind, aber im Wesentlichen ist es die Geschichte der erst
en '
        'Färinger und daher die wichtigste historische Quelle zur Wikingerz
eit '
        'auf den Färöern (siehe dort).\n'
        '# Chronologie\n'
        'Erstgenannte Jahreszahlen nach C. C. Rafn, in Klammern nach G. V.
C. '
        'Young\n'
        'Torkil Barfrost wird des Landes verwiesen\n'
        'Turið Torkilsdóttir wird geboren\n'
        'Ravnur Hólmgarðsfari kommt auf die Färöer und bringt Sigmundur und
'
        'Tóri nach Norwegen (Sigmundur und Tóri in Norwegen).\n'
        'Tóra Sigmundsdóttir (Sigmunds Tochter) wird geboren.\n'
        'Sigmundur und Tóri werden in das Gefolge von Hákon Jarl aufgenomme
n.\n'
        'Torkil Barfrost wird von der Verbannung freigesprochen\n'
        'Torkil wird Sysselmann in Orkedal',
 'en': 'It was written in Iceland shortly after 1200. The author is unknow
n '
        'and the original manuscript is lost to history, but passages of th
e '
        'original manuscript have been copied in other sagas, especially in
'
        'three manuscripts: Óláfs saga Tryggvasonar en mesta, Flateyjarbók,
and '
        'a manuscript registered as AM 62 fol.\n'
        'The different sagas differ somewhat on the first settlement of the
'
        'Faroes. Historians have understood from the beginning of Færeyinga
'
        'Saga in Flateyjarbók that Grímur Kamban settled in the Faroes when
'
        'Harald Fairhair was king of Norway (c.872-930 AD). This does not '
        'correspond with the writings of the Irish monk Dicuil.\n'
        'However, the version from Ólafs saga Tryggvasonar does correspond
with '
        'the writings of Dicuil. The opening text is, "There was a man name
d '
        'Grímr Kamban; he first settled in Faroe. But in the days of Harald
'
        'Fairhair many men fled before the king’s overbearing." The first m
an '
        'to settle in Faroe is, according to this text, a man with a Norse
'
        'first name and a Gaelic last name. This suggests that he might hav
e '
        'come from Norse–Gael settlements to the south in Scotland or Irela
nd '
        'and probably was not Norwegian.\n'
        'The text says that many men fled from Norway when Harald Fairhair
was '
        'king there, but it also says that the isles were settled before th
at '
        '(possibly for hundreds of years, although most historians do not t
hink '
```

```
        'so).\n'
        'The Norwegians who fled from Harald Fairhair must have known about
the '
        'isles before leaving Norway. If Grímr Kamban settled some time bef
ore, '
        'this could explain the Norwegians knowing about the Faroes.\n'
        'According to Dicuil, a Hiberno-Scottish mission, the "Papar", live
d in '
        'the Faroes (and Iceland) before Vikings came from Norway.',
 'es': 'El texto inicial es el siguiente:\n'
        'La saga se inicia con Þorbjörn Götuskeggr (apodo que recibe de su
'
        'asentamiento en Gøta (Eysturoy), padre de Þrándur Þorbjörnsson, el
'
        'personaje principal de la saga. Un hermano de Þrándr se llama '
        'Sigmundur de Skufoy y era padre dos hijos Brestir y Beinir, vasall
os '
        'del jarl de Lade Håkon Sigurdsson y dominan la mitad del archipiél
ago. '
        'La otra mitad está bajo el poder de Havgrímur de Suduroy, vasallo
de '
        'Harald II de Noruega. Þrándr de Gøta se las ingenia para conseguir
'
        'poder de los vasallos noruegos hasta el regreso de Sigmundur '
        'Brestisson de un largo exilio y grandes hazañas vikingas. Sigmundu
r '
        'protagoniza una venganza por la muerte de su padre Brestir, funda
un '
        'asentamiento en Skufoy y recibe la parte de las islas del jarl.\n'
        'La saga resalta la lucha por el poder de Þrándr y su clan familiar
por '
        'un lado, y por el otro Sigmundur y sus descendientes. Þrándr es pa
gano '
        'y fundamentado en el seidr, no se detiene ante nada para obtener '
        'riqueza y poder y es un implacable adversario de los noruegos. '
        'Sigmundur, por otro lado, es un noble guerrero y excepcional exper
to '
        'en armas que se convierte en vasallo de Olaf Tryggvason, es bautiz
ado, '
        'acepta la parte de las islas de la corona noruega y trae el '
        'cristianismo. La saga Færeyinga iguala en suspense y dramatismo a
las '
        'grandes sagas islandesas.',
 'it': 'La saga traccia la storia dei Faroensi dal loro insediamento nelle
'
        "isole fino alla conversione al Cristianesimo e l'annessione delle
Fær "
        'Øer nel Regno di Norvegia.\n'
        'Uno dei personaggi principali della saga è il norvegese Sigmundur
'
        "Brestisson (961 – 1005), che aveva ricevuto, nel 999, l'incarico d
a re "
        'Olaf I Tryggvason di convertire le isole al cristianesimo. Brestis
son '
        'era originario delle Fær Øer: la sua famiglia, infatti, era giunta
'
        "nell'isola attorno al IX secolo assieme ad altri esuli che fuggiva
no "
        'dalla tirannia di re Harald I Hårfagre, ma due secoli dopo, la '
        'famiglia di Brestisson, che aveva prosperato nelle isole meridiona
```

```
li '
       "dell'arcipelago, era stata sterminata dal contingente norvegese "
       'inviato da re Olaf I per conquistare le Fær Øer.\n'
       "Di fronte all'ostilità dei Faroensi alla conversione, Brestisson f
ece "
       'irruzione di notte, con un manipolo di uomini armati, nella reside
nza '
       'di Tróndur í Gøtu, capo dei Faroensi, e dopo averlo preso in custo
dia '
       'lo pose di fronte ad una scelta obbligata: o la conversione o la '
       'decapitazione. I Faroensi si convertirono.'}

--- summaries_translated ---
{'de': 'The Faroese saga begins with the words (in Faroese):\n'
       'Grímur Kamban was the first man to settle in the Faroe Islands.\n'
       "This was a day at Harald's Harfagra,\n"
       'Then a great multitude fled from before his face.\n'
       'in German:\n'
       'Grímur Kamban was the first man to settle in the Faroe Islands.\n'
       'This was in the days of Harald Fairhair,\n'
       'when many people were fleeing from his rule.\n'
       'It is believed that this Grímur actually lived and in the 9th cent
ury '
       'went from Norway to the Faroe Islands, settled in Funningur, and t
hus '
       'initiated the Norse takeover of the archipelago. However, this was
'
       'before the time of Harald Schönhaar, probably as early as 825. '
       'Obviously, the Färingersaga confuses this first wave of land-takin
g '
       'with the second wave around 885/890, which was actually an escape
from '
       "Harald Schönhaar's rule.\n"
       'A free republic of emigrants was established in the Faroe Islands,
and '
       'in their capital, Tórshavn, they had their own Ting. This place ca
lled '
       'Tinganes is still the seat of government today. The Faroese saga '
       'contains these and other historical facts, especially the story of
'
       'Sigmundur Brestisson (Sigmund), who Christianized the Faroe Island
s '
       "against the resistance of Trónd from Gøta (Trond) around 1000. Tro
nd's "
       'role is so central that the Färinger saga is also called The saga
of '
       'Trond.\n'
       'Since the tale was written by monks, it is not impartial, but it '
       'portrays Sigmund (the Christians) as the good and Trond (the pagan
s) '
       'as the bad. There are also parts of the Faroese saga that are '
       'obviously more fiction than truth, but in essence it is the story
of '
       'the first Faroese and therefore the most important historical sour
ce '
       'for the Viking Age in the Faroe Islands (see there).\n'
       '# Chronology\n'
       'First-named years after C. C. Rafn, in parentheses after G. V. C.
'
       'Young\n'
```

```
        'Torkil Barfrost expelled from the country\n'
        'Turið Torkilsdóttir is born\n'
        'Ravnur Hólmgarðsfari comes to the Faroe Islands and brings Sigmund
ur '
        'and Tóri to Norway (Sigmundur and Tóri in Norway).\n'
        "Tóra Sigmundsdóttir (Sigmund's daughter) is born.\n"
        'Sigmundur and Tóri are taken into the entourage of Hákon Jarl.\n'
        'Torkil Barfrost is acquitted of the banishment\n'
        'Torkil will become Sysselmann in Orkedal',
 'es': 'The opening text is as follows:\n'
        'The saga begins with Þorbjörn Götuskeggr (nickname he receives fro
m '
        'his settlement in Gøta (Eysturoy), father of Þrándur Þorbjörnsson,
the '
        'main character of the saga. A brother of Þrándr is called Sigmundu
r of '
        'Skufoy and he fathered two sons Brestir and Beinir, vassals of the
'
        'jarl of Lade Håkon Sigurdsson and dominate half of the archipelag
o. '
        'The other half is under the power of Havgrímur of Suduroy, vassal
of '
        'Harald II of Norway. Þrándr of Gøta managed to gain power from the
'
        'Norwegian vassals until the return of Sigmundur Brestisson from a
long '
        'exile and great Viking deeds. Sigmundur stages a revenge for the d
eath '
        'of his father Brestir, founds a settlement at Skufoy and receives
the '
        "jarl's share of the islands.\n"
        'The saga highlights the power struggle between Þrándr and his fami
ly '
        'clan on the one hand, and Sigmundur and his descendants on the oth
er. '
        'Þrándr is pagan and seidr-based, stops at nothing to gain wealth a
nd '
        'power and is an implacable adversary of the Norwegians. Sigmundur,
on '
        'the other hand, is a noble warrior and exceptional weapons expert
who '
        "becomes Olaf Tryggvason's vassal, is baptized, accepts the island
s' "
        'share of the Norwegian crown, and brings Christianity. The Færeyin
ga '
        'saga is equal in suspense and drama to the great Icelandic saga
s.',
 'it': 'The saga traces the history of the Faroese from their settlement i
n '
        'the islands to their conversion to Christianity and the annexation
of '
        'the Fær Øer to the Kingdom of Norway.\n'
        'One of the main characters of the saga is the Norwegian Sigmundur
'
        'Brestisson (961  1005), who had received, in 999, the commission f
rom '
        'King Olaf I Tryggvason to convert the islands to Christianity. '
        'Brestisson was originally from the Fær Øer: his family, in fact, h
ad '
        'come to the island around the 9th century along with other exiles
```

```
'
       'fleeing the tyranny of King Harald I Hårfagre, but two centuries '
       "later, Brestisson's family, who had prospered in the southern isla
nds "
       'of the archipelago, had been exterminated by the Norwegian conting
ent '
       'sent by King Olaf I to conquer the Fær Øer.\n'
       'Faced with the hostility of the Faroese to the conversion, Brestis
son '
       'made a night raid, with a handful of armed men, on the residence o
f '
       'Tróndur í Gøtu, head of the Faroese, and after taking him into cus
tody '
       'he put him in front of a forced choice: either conversion or '
       'beheading. The Faroese converted.'}

--- summaries_anonymized ---
{'de': 'Entity H begins with the words (in Entity Q):\n'
       'Cora was the first man to settle in Location C.\n'
       "This was a day at Kenneth's Location O,\n"
       "Then a great multitude fled from before Kenneth's face.\n"
       'in Entity W:\n'
       'Cora was the first man to settle in Location C.\n'
       'This was in the days of Kenneth,\n'
       "when many people were fleeing from Kenneth's rule.\n"
       'It is believed that Cora actually lived and in the 9th century wen
t '
       'from Location L to Location C, settled in Location H, and thus '
       'initiated the Entity U takeover of Location C. However, this was '
       'before the time of Elise, probably as early as 825. Obviously, Ent
ity '
       'J confuses this first wave of land-taking with the second wave aro
und '
       "885/890, which was actually an escape from Elise's rule.\n"
       'A free republic of emigrants was established in Location C, and in
'
       'their capital, Location D, they had their own Ting. Wilma is still
the '
       'seat of government today. Entity H contains these and other histor
ical '
       'facts, especially the story of Angela (Robert), who Christianized
'
       "Location C against the resistance of Ida around 1000. Ida's role i
s so "
       'central that Entity J is also called The saga of Ida.\n'
       'Since the tale was written by monks, it is not impartial, but it '
       'portrays Robert (the Entity Q) as the good and Ida (the pagans) as
the '
       'bad. There are also parts of Entity H that are obviously more fict
ion '
       'than truth, but in essence Entity H is the story of the first Enti
ty Q '
       'and therefore the most important historical source for the Entity
B in '
       'Location C (see there).\n'
       '# Chronology\n'
       'First-named years after C. Charlotte, in parentheses after G. V.
C. '
       'Jennifer expelled from the country\n'
       'Organization N is born\n'
```

```
        'Eric comes to Location C and brings Location M to Location L (Loca
tion '
        'M in Location L).\n'
        "David (Robert's daughter) is born.\n"
        'Location M are taken into the entourage of Jean.\n'
        'Jennifer is acquitted of the banishment\n'
        'Jennifer will become Oscar in Location H',
 'en': 'It was written in Location O shortly after 1200. The author is unk
nown '
        'and the original manuscript is lost to history, but passages of th
e '
        'original manuscript have been copied in other sagas, especially in
'
        'three manuscripts: Entity E, Entity B, and a manuscript registered
as '
        'AM 62 fol.\n'
        'The different sagas differ somewhat on the first settlement of '
        'Location C. Historians have understood from the beginning of Entit
y C '
        'in Entity B that Troy settled in Location C when Ralph was king of
'
        'Location K (c.872–930 AD). This does not correspond with Amand
a.\n'
        'However, the version from Entity E does correspond with Amanda. Th
e '
        'opening text is, "There was Troy; he first settled in Location C.
But '
        'in the days of Ralph many men fled before Ralph overbearing." Troy
is, '
        'according to this text, a man with a Entity I first name and a Ent
ity '
        'F last name. This suggests that Troy might have come from Entity '
        'I–Location M settlements to the south in Location Y or Location L
and '
        'probably was not Entity P.\n'
        'The text says that many men fled from Location K when Ralph was ki
ng '
        'there, but it also says that Location C were settled before that '
        '(possibly for hundreds of years, although most historians do not t
hink '
        'so).\n'
        'Entity N must have known about Location C before leaving Location
K. '
        'If Troy settled some time before, this could explain Entity N know
ing '
        'about Location C.\n'
        'According to Jody, a Entity I mission, the "Entity T", lived in '
        'Location C (and Location O) before Organization P came from Locati
on '
        'K.',
 'es': 'The opening text is as follows:\n'
        'Kathleen begins with Mary (nickname Nina receives from his settlem
ent '
        'in Location C, father of June, the main character of Kathleen. A '
        'brother of Nina is called Louise of Location F and Kent fathered t
wo '
        'sons James and Danny, vassals of Stephanie and dominate half of th
e '
        'archipelago. The other half is under the power of Location M of '
        'Location G, vassal of Anya of Location A. Nina managed to gain pow
```

```
er '
       'from the Entity L vassals until the return of Kent from a long exi
le '
       'and great Entity W deeds. Kent stages a revenge for the death of h
is '
       'father Brestir, founds a settlement at Location F and receives '
       "Stephanie's share of the islands.\n"
       'Kathleen highlights the power struggle between Nina and his family
'
       'clan on the one hand, and Kent and his descendants on the other. N
ina '
       'is pagan and seidr-based, stops at nothing to gain wealth and powe
r '
       'and is an implacable adversary of the Entity A. Kent, on the other
'
       'hand, is a noble warrior and exceptional weapons expert who become
s '
       "Ronald's vassal, is baptized, accepts the islands' share of the "
       'Norwegian crown, and brings Entity S. Kathleen is equal in suspens
e '
       'and drama to the great Entity G sagas.',
 'it': 'The saga traces the history of Entity A from their settlement in t
he '
       'islands to their conversion to Entity D and the annexation of the
Fær '
       'Øer to the Location O.\n'
       'Philip is the Entity K Tarah (961  1005), who had received, in 99
9, '
       'the commission from Milton to convert the islands to Entity D. Phi
lip '
       'was originally from the Fær Øer: his family, in fact, had come to
the '
       'island around the 9th century along with other exiles fleeing the
'
       "tyranny of King John, but two centuries later, Philip's family, wh
o "
       'had prospered in the southern islands of the archipelago, had been
'
       'exterminated by the Entity K contingent sent by Milton to conquer
the '
       'Fær Øer.\n'
       'Faced with the hostility of Entity A to the conversion, Philip mad
e a '
       'night raid, with a handful of armed men, on the residence of Lisa,
and '
       'after taking Lisa into custody Philip put him in front of a forced
'
       'choice: either conversion or beheading. Entity A converted.'}

--- num_sentences ---
{'de': 26, 'en': 14, 'es': 10, 'it': 5}

--- sentences ---
{'de': ['The Faroese saga begins with the words (in Faroese):',
        'Grímur Kamban was the first man to settle in the Faroe Islands.',
        "This was a day at Harald's Harfagra,",
        'Then a great multitude fled from before his face.',
        'in German:',
        'Grímur Kamban was the first man to settle in the Faroe Islands.',
        'This was in the days of Harald Fairhair,',
```

```
        'when many people were fleeing from his rule.',
        'It is believed that this Grímur actually lived and in the 9th cen
tury '
        'went from Norway to the Faroe Islands, settled in Funningur, and
thus '
        'initiated the Norse takeover of the archipelago.',
        'However, this was before the time of Harald Schönhaar, probably a
s '
        'early as 825.',
        'Obviously, the Färingersaga confuses this first wave of land-taki
ng '
        'with the second wave around 885/890, which was actually an escape
'
        "from Harald Schönhaar's rule.",
        'A free republic of emigrants was established in the Faroe Island
s, '
        'and in their capital, Tórshavn, they had their own Ting.',
        'This place called Tinganes is still the seat of government toda
y.',
        'The Faroese saga contains these and other historical facts, '
        'especially the story of Sigmundur Brestisson (Sigmund), who '
        'Christianized the Faroe Islands against the resistance of Trónd f
rom '
        'Gøta (Trond) around 1000.',
        "Trond's role is so central that the Färinger saga is also called
The "
        'saga of Trond.',
        'Since the tale was written by monks, it is not impartial, but it
'
        'portrays Sigmund (the Christians) as the good and Trond (the paga
ns) '
        'as the bad.',
        'There are also parts of the Faroese saga that are obviously more
'
        'fiction than truth, but in essence it is the story of the first '
        'Faroese and therefore the most important historical source for th
e '
        'Viking Age in the Faroe Islands (see there).',
        '# Chronology',
        'First-named years after C. C. Rafn, in parentheses after G. V. C.
'
        'Young',
        'Torkil Barfrost expelled from the country',
        'Turið Torkilsdóttir is born',
        'Ravnur Hólmgarðsfari comes to the Faroe Islands and brings Sigmun
dur '
        'and Tóri to Norway (Sigmundur and Tóri in Norway).',
        "Tóra Sigmundsdóttir (Sigmund's daughter) is born.",
        'Sigmundur and Tóri are taken into the entourage of Hákon Jarl.',
        'Torkil Barfrost is acquitted of the banishment',
        'Torkil will become Sysselmann in Orkedal'],
 'en': ['It was written in Iceland shortly after 1200.',
        'The author is unknown and the original manuscript is lost to hist
ory, '
        'but passages of the original manuscript have been copied in other
'
        'sagas, especially in three manuscripts: Óláfs saga Tryggvasonar e
n '
        'mesta, Flateyjarbók, and a manuscript registered as AM 62 fol.',
        'The different sagas differ somewhat on the first settlement of th
```

```
e '
        'Faroes.',
        'Historians have understood from the beginning of Færeyinga Saga i
n '
        'Flateyjarbók that Grímur Kamban settled in the Faroes when Harald
'
        'Fairhair was king of Norway (c.872-930 AD).',
        'This does not correspond with the writings of the Irish monk Dicu
il.',
        'However, the version from Ólafs saga Tryggvasonar does correspond
'
        'with the writings of Dicuil.',
        'The opening text is, "There was a man named Grímr Kamban; he firs
t '
        'settled in Faroe.',
        'But in the days of Harald Fairhair many men fled before the kin
g’s '
        'overbearing."',
        'The first man to settle in Faroe is, according to this text, a ma
n '
        'with a Norse first name and a Gaelic last name.',
        'This suggests that he might have come from Norse-Gael settlements
to '
        'the south in Scotland or Ireland and probably was not Norwegia
n.',
        'The text says that many men fled from Norway when Harald Fairhair
was '
        'king there, but it also says that the isles were settled before t
hat '
        '(possibly for hundreds of years, although most historians do not
'
        'think so).',
        'The Norwegians who fled from Harald Fairhair must have known abou
t '
        'the isles before leaving Norway.',
        'If Grímr Kamban settled some time before, this could explain the
'
        'Norwegians knowing about the Faroes.',
        'According to Dicuil, a Hiberno-Scottish mission, the "Papar", liv
ed '
        'in the Faroes (and Iceland) before Vikings came from Norway.'],
 'es': ['The opening text is as follows:',
        'The saga begins with Þorbjörn Götuskeggr (nickname he receives fr
om '
        'his settlement in Gøta (Eysturoy), father of Þrándur Þorbjörnsso
n, '
        'the main character of the saga.',
        'A brother of Þrándr is called Sigmundur of Skufoy and he fathered
two '
        'sons Brestir and Beinir, vassals of the jarl of Lade Håkon Sigurd
sson '
        'and dominate half of the archipelago.',
        'The other half is under the power of Havgrímur of Suduroy, vassal
of '
        'Harald II of Norway.',
        'Þrándr of Gøta managed to gain power from the Norwegian vassals u
ntil '
        'the return of Sigmundur Brestisson from a long exile and great Vi
king '
        'deeds.',
```

```
        'Sigmundur stages a revenge for the death of his father Brestir, '
        "founds a settlement at Skufoy and receives the jarl's share of th
e "
        'islands.',
        'The saga highlights the power struggle between Þrándr and his fam
ily '
        'clan on the one hand, and Sigmundur and his descendants on the ot
her.',
        'Þrándr is pagan and seidr-based, stops at nothing to gain wealth
and '
        'power and is an implacable adversary of the Norwegians.',
        'Sigmundur, on the other hand, is a noble warrior and exceptional
'
        "weapons expert who becomes Olaf Tryggvason's vassal, is baptized,
"
        "accepts the islands' share of the Norwegian crown, and brings "
        'Christianity.',
        'The Færeyinga saga is equal in suspense and drama to the great '
        'Icelandic sagas.'],
 'it': ['The saga traces the history of the Faroese from their settlement
in '
        'the islands to their conversion to Christianity and the annexatio
n of '
        'the Fær Øer to the Kingdom of Norway.',
        'One of the main characters of the saga is the Norwegian Sigmundur
'
        'Brestisson (961 1005), who had received, in 999, the commission f
rom '
        'King Olaf I Tryggvason to convert the islands to Christianity.',
        'Brestisson was originally from the Fær Øer: his family, in fact,
had '
        'come to the island around the 9th century along with other exiles
'
        'fleeing the tyranny of King Harald I Hårfagre, but two centuries
'
        "later, Brestisson's family, who had prospered in the southern isl
ands "
        'of the archipelago, had been exterminated by the Norwegian contin
gent '
        'sent by King Olaf I to conquer the Fær Øer.',
        'Faced with the hostility of the Faroese to the conversion, Bresti
sson '
        'made a night raid, with a handful of armed men, on the residence
of '
        'Tróndur í Gøtu, head of the Faroese, and after taking him into '
        'custody he put him in front of a forced choice: either conversion
or '
        'beheading.',
        'The Faroese converted.']}

--- genres ---
[]

--- similarities_labels ---
['en', 'de', 'it', 'es']

--- similarities shape ---
(4, 4)
```

Analysis 3: Inspecting Stories with No English original but have >=2 EN summaries

This is the case where stories do not have an original English summary, but there are at least two English summaries available from translations or other sources. In the next cell, we identify such stories and explain what should be kept or removed.

Results:

- These stories can be kept for retrieval task.
- Delete the ones that have no EN original, but also don't have >=2 EN summaries.

In [10]:
```
# Inspect stories that truly have no English original summary (not just b
no_en_stories = [s for s in storyDataset if "en" not in s.summaries_origi
print(f"\nThere are {len(no_en_stories)} stories with NO English original

# Find how many of those have >=2 EN summaries (across all sources, inclu
no_en_multiple_en = [
    s for s in no_en_stories if len(s.get_all_summaries_en()[0]) >= 2
]
print(f"Out of these, {len(no_en_multiple_en)} stories have >=2 EN summar

# Optionally: show an example and explore their EN (translated) deduped s
example_no_en = None
for s in no_en_multiple_en:
    example_no_en = s
    break

if example_no_en is not None:
    print("\033[1;33m\nExample story with no native EN but multiple EN su
    fields = [
        "wikidata_id", "title", "titles", "description",
        "summaries_original", "summaries_translated", "summaries_anonymiz
        "num_sentences", "sentences", "genres", "similarities_labels"
    ]
    for field in fields:
        print(f"\n--- {field} ---")
        value = getattr(example_no_en, field, None)
        if isinstance(value, (dict, list)):
            pprint.pprint(value)
        else:
            print(value)

    print("\n--- similarities shape ---")
    similarities = getattr(example_no_en, "similarities", None)
    if hasattr(similarities, "shape"):
        print(similarities.shape)
    else:
        print(type(similarities))

    ids_en, summaries_en = example_no_en.get_all_summaries_en()
    print(f"\n--- get_all_summaries_en() ---")
    print(f"  {len(ids_en)} EN summaries (ids: {ids_en})")
else:
    print("No example with multiple EN summaries found among stories lack
```

```
There are 3120 stories with NO English original summary.
Out of these, 2698 stories have >=2 EN summaries (after deduplication, acr
oss all sources).
```

Example story with no native EN but multiple EN summaries (from translatio ns only):

```
--- wikidata_id ---
Q100144601

--- title ---
Summer White

--- titles ---
{'de': 'Blanco de Verano', 'fr': 'Summer White'}

--- description ---
film

--- summaries_original ---
{'de': 'Der 13-jährige Rodrigo ist der kleine Prinz seiner Mutter Valeria.
Die '
       'beiden verbindet eine innige Beziehung. Bislang haben die beiden '
       'alleine in ihrem kleinen Haus am Stadtrand von Mexiko-Stadt geleb
t. '
       'Die Dinge ändern sich, als ihr neuer Freund Fernando bei ihnen '
       'einzieht und Rodrigos Mutter versucht, sich ein neues Leben mit ih
m '
       'aufzubauen. Rodrigo ist hin- und hergerissen, ob er Fernando als T
eil '
       'einer neuen Familie akzeptieren kann oder ob er um den Platz an de
r '
       'Seite seiner Mutter kämpfen soll und damit vielleicht das neue Glü
ck '
       'des Menschen zu zerstören, den er am meisten liebt.\n'
       'Eigentlich ist Fernando ein netter Kerl, doch Rodrigo weiß nicht s
o '
       'recht, ob er seinen Vorschlag, die farbigen Wände in ihrem Haus we
iß '
       'zu streichen, wirklich mag. Er beginnt, sich immer öfter aus dem H
aus '
       'zu verziehen und Joints zu rauchen. Valeria droht damit, Rodrigo z
u '
       'seinem Vater zu schicken, was für ihren Sohn die schlimmste Art vo
n '
       'Bestrafung wäre, die er sich vorstellen kann.',
 'fr': 'Rodrigo, un jeune adolescent solitaire en fusion avec sa mère, vit
'
       "difficilement l'arrivée d'un nouveau compagnon pour celle-ci."}

--- summaries_translated ---
{'de': "Thirteen-year-old Rodrigo is his mother Valeria's little prince. T
he "
       'two share an intimate relationship. So far, the two have lived alo
ne '
       'in their small house on the outskirts of Mexico City. Things chang
e '
       "when her new boyfriend Fernando moves in with them and Rodrigo's "
       'mother tries to build a new life with him. Rodrigo is torn between
'
```

```
       'whether to accept Fernando as part of a new family or whether to f
ight '
       "for his place at his mother's side, perhaps destroying the new "
       'happiness of the person he loves most.\n'
       "Actually, Fernando is a nice guy, but Rodrigo isn't sure if he rea
lly "
       'likes his suggestion to paint the colored walls in her house whit
e. He '
       'starts sneaking out of the house more and more and smoking joints.
'
       'Valeria threatens to send Rodrigo to his father, which would be th
e '
       'worst kind of punishment imaginable for her son.',
 'fr': 'Rodrigo, a lonely teenager who has become one with his mother, fin
ds '
       'it difficult to accept the arrival of a new companion for her.'}

--- summaries_anonymized ---
{'de': "Daniel is his mother Valeria's little prince. The two share an "
       'intimate relationship. So far, the two have lived alone in their s
mall '
       "house on the outskirts of Mexico City. Things change when Billie's
new "
       "boyfriend Fernando moves in with them and Daniel's mother tries to
"
       'build a new life with Julio. Daniel is torn between whether to acc
ept '
       "Julio as part of a new family or whether to fight for Daniel's pla
ce "
       "at his mother's side, perhaps destroying the new happiness of Juli
o.\n"
       "Actually, Julio is a nice guy, but Daniel isn't sure if he really
"
       "likes Julio's suggestion to paint the colored walls in Billie's ho
use "
       'white. Daniel starts sneaking out of Location E more and more and
'
       'smoking joints. Billie threatens to send Daniel to his father, whi
ch '
       'would be the worst kind of punishment imaginable for Daniel.',
 'fr': 'Mary, finds it difficult to accept the arrival of a new companion
for '
       'her.'}

--- num_sentences ---
{'de': 8, 'fr': 1}

--- sentences ---
{'de': ["Thirteen-year-old Rodrigo is his mother Valeria's little princ
e.",
        'The two share an intimate relationship.',
        'So far, the two have lived alone in their small house on the '
        'outskirts of Mexico City.',
        'Things change when her new boyfriend Fernando moves in with them
and '
        "Rodrigo's mother tries to build a new life with him.",
        'Rodrigo is torn between whether to accept Fernando as part of a n
ew '
        "family or whether to fight for his place at his mother's side, "
        'perhaps destroying the new happiness of the person he loves mos
```

```
t.',
        "Actually, Fernando is a nice guy, but Rodrigo isn't sure if he re
ally "
        'likes his suggestion to paint the colored walls in her house whit
e.',
        'He starts sneaking out of the house more and more and smoking joi
nts.',
        'Valeria threatens to send Rodrigo to his father, which would be t
he '
        'worst kind of punishment imaginable for her son.'],
 'fr': ['Rodrigo, a lonely teenager who has become one with his mother, fi
nds '
        'it difficult to accept the arrival of a new companion for her.']}

--- genres ---
['Q130232']

--- similarities_labels ---
['de', 'fr']

--- similarities shape ---
(2, 2)

--- get_all_summaries_en() ---
  2 EN summaries (ids: ['fr', 'de'])
```

Analysis 4: Inspecting stories with no translated summaries

Here we inspect stories that lack an original English summary, but have multiple deduplicated English summaries originating from translations or other sources. Below, we identify such stories, show an example, and examine the relevant fields.

Results:

- Remove this story from the final dataset.

In [11]:
```
# Inspect how many stories have no translated summaries
no_summaries_translated = sum(1 for s in storyDataset if not s.summaries_
print(f"\nThere are {no_summaries_translated} stories with NO translated

if no_summaries_translated > 0:
    # Show an example story with no translated summaries
    example_no_translated = next(s for s in storyDataset if not s.summari
    print("\033[1;33m\nExample story with no translated summaries:\033[0m
    fields = [
        "wikidata_id", "title", "titles", "description",
        "summaries_original", "summaries_translated", "summaries_anonymiz
        "num_sentences", "sentences", "genres", "similarities_labels"
    ]
    for field in fields:
        print(f"\n--- {field} ---")
        value = getattr(example_no_translated, field, None)
        if isinstance(value, (dict, list)):
            import pprint
            pprint.pprint(value)
        else:
            print(value)
```

```
    print("\n--- similarities shape ---")
    similarities = getattr(example_no_translated, "similarities", None)
    if hasattr(similarities, "shape"):
        print(similarities.shape)
    else:
        print(type(similarities))
else:
    print("No stories without translated summaries found.")
```

```
There are 1 stories with NO translated summaries.

Example story with no translated summaries:

--- wikidata_id ---
Q997766

--- title ---
Book of Odes

--- titles ---
{'en': 'Book of Odes', 'it': 'Odi'}

--- description ---
book

--- summaries_original ---
{'en': 'Chapters of this book as presented by Rahlfs are:', 'it': ''}

--- summaries_translated ---
{}

--- summaries_anonymized ---
{'en': 'Chapters of this book as presented by Entity Z are:'}

--- num_sentences ---
{'en': 1}

--- sentences ---
{'en': ['Chapters of this book as presented by Rahlfs are:']}

--- genres ---
[]

--- similarities_labels ---
['en']

--- similarities shape ---
(1, 1)
```

Analysis 5: Inspecting stories with no blank description

We inspect stories with a blank or missing description, count how many there are, and show an example with relevant fields.

Results:

- Since description is not a key variable in our pipline, these stories can be kept.

- However, if stratified sampling will be done, and description is part of it, then remove those

In [12]:
```
# Inspect how many stories have a blank (missing or just whitespace) desc
no_description = sum(1 for s in storyDataset if not (s.description or "")
print(f"\nThere are {no_description} stories with BLANK description.")

if no_description > 0:
    # Show an example story with a blank description
    example_blank_desc = next(s for s in storyDataset if not (s.descripti
    print("\033[1;33m\nExample story with blank description:\033[0m")
    fields = [
        "wikidata_id", "title", "titles", "description",
        "summaries_original", "summaries_translated", "summaries_anonymiz
        "num_sentences", "sentences", "genres", "similarities_labels"
    ]
    for field in fields:
        print(f"\n--- {field} ---")
        value = getattr(example_blank_desc, field, None)
        if isinstance(value, (dict, list)):
            import pprint
            pprint.pprint(value)
        else:
            print(value)

    print("\n--- similarities shape ---")
    similarities = getattr(example_blank_desc, "similarities", None)
    if hasattr(similarities, "shape"):
        print(similarities.shape)
    else:
        print(type(similarities))
else:
    print("No stories with blank description found.")
```

```
There are 32 stories with BLANK description.
```

Example story with blank description:

```
--- wikidata_id ---
Q1033567

--- title ---
Canon Episcopi

--- titles ---
{'de': 'Canon episcopi', 'it': 'Canon episcopi'}

--- description ---
None

--- summaries_original ---
{'de': 'Wegen ihrer Abweichung vom christlichen Glauben der Kirche bezeich
net '
       'der Kanon Wahrsager und Zauberer, seien es Männer oder Frauen, als
'
       'Häretiker (Ketzer). Die Bischöfe und ihre Helfer, d. h. der '
       'Pfarrklerus, sollen die bekämpfen, welche den Glauben an Hexen '
       'verbreiteten und die überführten Personen verstoßen. Die vom Kanon
'
       'geforderte Ausweisung der Ketzer aus dem Pfarrbezirk hat als '
       'ausgesprochen schwere Kirchenstrafe zu gelten.\n'
       'Als offensichtlich besonders schwere Fälle werden in diesem Rechts
text '
       'jene „frevelhaften Frauen“ bezeichnet, die sich von Gott wieder Sa
tan '
       'zugewandt hätten und die von sich behaupten oder daran glauben wür
den, '
       'zusammen mit einer großen Anzahl anderer Frauen im Gefolge der '
       'heidnischen Göttin Diana auf Tieren nächtliche Flüge über große '
       'Strecken hinweg unternommen zu haben. Diese Frauen, so der Kanon,
'
       'würden der Göttin Diana wie einer Herrin gehorchen und in bestimmt
en '
       'Nächten von ihr herbeigerufen werden. Diese Gefolgschaft der besag
ten '
       'Frauen gegenüber ihrer Herrin Diana und deren Verehrung als göttli
ches '
       'Wesen wird vom Kanon als Treulosigkeit bzw. Frevel gegen Gott, Abf
all '
       'vom rechten Glauben und Rückfall ins Heidentum gedeutet.\n'
       'Der Fehler dieser Frauen bestand also nicht darin, dass sie Hexen
'
       'waren, sondern dass sie behaupteten es zu sein, obwohl sie es nich
t '
       'waren. Der Canon Episcopi ist eine Verurteilung des Hexenglaubens.
Der '
       'Canon Episcopi legt fest, dass der Hexenwahn teuflisch sei.\n'
       'Der zentrale Satz lautet:\n'
       'Die Glaubensvorstellung der Frauen, gegen die auch die Priester mi
t '
       'allem Nachdruck predigen sollen, wird im weiteren Text als Einbild
ung '
       'und Wahnvorstellung bezeichnet. Satan, der sich in einen Engel des
'
```

```
        'Lichts verwandelt habe, würde den Frauen solche Wahngebilde '
        'vorspiegeln, um auf diese Weise bei den Menschen den Unglauben zu
'
        'verbreiten. Er könne sich in die Gestalten wie auch in die Abbilde
r '
        'beliebiger Personen verwandeln und die Frauen auf diese Weise in i
hren '
        'Träumen täuschen. Diese Frauen würden nun glauben, diese Einbildun
g '
        'der Luftfahrt auch körperlich erfahren zu haben. So die Deutung de
r '
        'Vorstellung der Frauen aus der Perspektive des frühmittelalterlich
en '
        'Kirchenrechts. Ob sich hinter den bekämpften Vorstellungen ein in
'
        'verschiedensten Formen in Europa verbreiteter heidnisch-christlich
er '
        'Feenglauben verbirgt, ist umstritten.',
 'it': 'Il Canon episcopi definiva la stregoneria "culto verso il demoni
o", ma '
        'negava che le streghe potessero volare fisicamente e dichiarava ch
e '
        '«[...]chiunque è così stupido e folle da credere a storie tanto '
        'fantasiose è da considerarsi un infedele, perché ciò deriva da '
        "un'illusione del demonio». Tali voli notturni, ritenuti materialme
nte "
        'impossibili e ritenuti illusori, potevano però realizzarsi con lo
'
        "spirito e «pur volando con lo spirito e l'immaginazione, queste "
        'streghe sono ugualmente colpevoli, come se lo avessero fatto in ca
rne '
        'e ossa».\n'
        'Nel XII secolo il Canon episcopi creò non pochi problemi ai '
        'demonologi, per via delle sue interne contraddizioni: a partire da
'
        'queste deduzioni, infatti, risultava semplice affermare che tutti
gli '
        "eretici e le streghe (fisicamente o con l'immaginazione) avessero
"
        'stretto un patto con il diavolo. Inoltre non era possibile difende
rsi '
        'da eventuali accuse, poiché nessuno poteva verosimilmente dimostra
re '
        'cosa avesse immaginato o meno.\n'
        'Una differente lettura del Canon episcopi ne fa un documento di '
        'moderazione, che riduce la stregoneria a pura vanità, esecrabile m
a '
        "punibile con provvedimenti disciplinari come l'allontanamento dall
a "
        'comunità dei credenti. Questa intuizione, che di fatto guidava gli
'
        'interventi dei vescovi (in altra maniera procedevano le autorità '
        'civili), non venne più seguita dalla fine del Quattrocento. Verso
la '
        'metà del XV secolo la maggioranza degli inquisitori e dei demonolo
gi '
        'cominciò a trascurare il Canon episcopi a motivo della sua '
        'problematicità e preferì utilizzare nuovi e più efficienti manuali
'
        'inquisitoriali, tra cui il più diffuso fu il Malleus Maleficarum
```

```
(Il '
       'Martello delle streghe).'}

--- summaries_translated ---
{'de': 'Because of their deviation from the Christian faith of the Church,
the '
       'canon designates diviners and sorcerers, whether men or women, as
'
       'heretics. The bishops and their helpers, that is, the parish clerg
y, '
       'are to fight those who spread the belief in witches and expel thos
e '
       'who are convicted. The expulsion of heretics from the parish, as '
       'required by the canon, is to be regarded as a very severe '
       'ecclesiastical penalty.\n'
       'Obviously particularly serious cases in this legal text are those
of '
       '"rebellious women" who would have turned from God back to Satan an
d '
       'who would claim or believe that they, along with a large number of
'
       'other women, were following the pagan goddess Diana in making '
       'long-distance night flights on animals. These women, according to
the '
       'canon, would obey the goddess Diana like a mistress and be summone
d by '
       'her on certain nights. This obedience of the aforementioned women
to '
       'their mistress Diana and their worship of her as a divine being is
'
       'interpreted by the Canon as unfaithfulness or rebellion against Go
d, '
       'apostasy from the true faith and reversion to paganism.\n'
       'So the fault of these women was not that they were witches, but th
at '
       'they claimed to be witches when they were not. The Canon Episcopi
is a '
       'condemnation of the belief in witchcraft. The Canon Episcopi decla
res '
       'that witchcraft is diabolical.\n'
       'The key phrase is:\n'
       "The idea of women's faith, against which the priests are also to "
       'preach with all emphasis, is described in the rest of the text as
'
       'imagination and delusion. Satan, who had transformed himself into
an '
       'angel of light, would project such delusions upon women in order t
o '
       'spread unbelief among men. He could transform himself into the sha
pes '
       'and images of any person and thus deceive the women in their dream
s. '
       'These women would now believe that they had also physically '
       'experienced this illusion of aviation. So the interpretation of th
e '
       'idea of women from the perspective of early medieval canon law. It
is '
       'debatable whether the ideas being challenged conceal a pagan-Chris
tian '
       'belief in fairies that is widespread in Europe in various forms.',
```

```
 'it': 'The Canon episcopal defined witchcraft as "worship of the devil",
but '
       'denied that witches could fly physically and declared that "anyone
who '
       'is so stupid and foolish as to believe such fanciful stories is to
be '
       'considered an infidel, because it is derived from an illusion of t
he '
       'devil". Such night flights, considered materially impossible and '
       'considered illusory, could however be realized with the spirit and
'
       'although flying with the spirit and imagination, these witches are
'
       'equally guilty, as if they had done it in flesh and blood.\n'
       'In the 12th century the Canon episcopal created not a few problems
for '
       'demonologists, because of its internal contradictions: starting fr
om '
       'these deductions, in fact, it was easy to affirm that all heretics
and '
       'witches (physically or imagination) had made a pact with the devi
l. '
       'Moreover, it was not possible to defend oneself against possible '
       'charges, since no one could plausibly prove what he had imagined o
r '
       'not.\n'
       'A different reading of the Canon of Bishops makes it a document of
'
       'moderation, which reduces witchcraft to pure vanity, execrable but
'
       'punishable by disciplinary measures such as expulsion from the '
       'community of believers. This intuition, which in fact guided the '
       'interventions of the bishops (otherwise the civil authorities '
       'proceeded), was no longer followed by the end of the fifteenth '
       'century. By the middle of the 15th century, the majority of '
       "inquisitors and demonologists began to neglect the Bishops' Canon
"
       'because of its problematic nature and preferred to use new and mor
e '
       'efficient inquisitorial manuals, the most widespread of which was
the '
       'Malleus Maleficarum (The Hammer of Witches).'}

--- summaries_anonymized ---
{'de': 'Because of their deviation from the Entity J faith of the Organiza
tion '
       'Q, Entity B designates diviners and sorcerers, whether men or wome
n, '
       'as heretics. The bishops and their helpers, that is, the parish '
       'clergy, are to fight those who spread the belief in witches and ex
pel '
       'those who are convicted. The expulsion of heretics from the paris
h, as '
       'required by Entity B, is to be regarded as a very severe '
       'ecclesiastical penalty.\n'
       'Obviously particularly serious cases in Entity B are those of '
       '"rebellious women" who would have turned from Frances back to Enti
ty F '
       'and who would claim or believe that they, along with a large numbe
r of '
```

```
        'other women, were following Ruth in making long-distance night fli
ghts '
        'on animals. These women, according to Entity B, would obey Ruth li
ke a '
        "mistress and be summoned by Ruth's on certain nights. This obedien
ce "
        'of the aforementioned women to their mistress Diana and their wors
hip '
        'of her as a divine being is interpreted by Entity B as unfaithfuln
ess '
        'or rebellion against Frances, apostasy from the true faith and '
        'reversion to paganism.\n'
        'So the fault of these women was not that they were witches, but th
at '
        'they claimed to be witches when they were not. Entity B is a '
        'condemnation of the belief in witchcraft. Entity B declares that '
        'witchcraft is diabolical.\n'
        'The key phrase is:\n'
        "The idea of women's faith, against which the priests are also to "
        'preach with all emphasis, is described in the rest of Entity B as
'
        'imagination and delusion. Entity F, would project such delusions u
pon '
        'women in order to spread unbelief among men. Entity F could transf
orm '
        'Entity F into the shapes and images of any person and thus deceive
the '
        'women in their dreams. These women would now believe that they had
'
        'also physically experienced this illusion of aviation. So the '
        'interpretation of the idea of women from the perspective of early
'
        'medieval canon law. It is debatable whether the ideas being challe
nged '
        'conceal a Entity X belief in fairies that is widespread in Locatio
n X '
        'in various forms.',
 'it': 'Entity E defined witchcraft as "worship of the devil", but denied
that '
        'witches could fly physically and declared that "anyone who is so '
        'stupid and foolish as to believe such fanciful stories is to be '
        'considered an infidel, because it is derived from an illusion of t
he '
        'devil". Such night flights, considered materially impossible and '
        'considered illusory, could however be realized with the spirit and
'
        'although flying with the spirit and imagination, these witches are
'
        'equally guilty, as if they had done it in flesh and blood.\n'
        'In the 12th century Entity E created not a few problems for '
        "demonologists, because of Entity E's internal contradictions: star
ting "
        'from these deductions, in fact, it was easy to affirm that all '
        'heretics and witches (physically or imagination) had made a pact w
ith '
        'the devil. Moreover, it was not possible to defend oneself against
'
        'possible charges, since no one could plausibly prove what he had '
        'imagined or not.\n'
        'A different reading of Entity E makes it a document of moderation,
```

```
'
       'which reduces witchcraft to pure vanity, execrable but punishable
by '
       'disciplinary measures such as expulsion from the community of '
       'believers. This intuition, which in fact guided the interventions
of '
       'the bishops (otherwise the civil authorities proceeded), was no lo
nger '
       'followed by the end of the fifteenth century. By the middle of the
'
       '15th century, the majority of inquisitors and demonologists began
to '
       'neglect Entity E because of its problematic nature and preferred t
o '
       'use new and more efficient inquisitorial manuals, the most widespr
ead '
       'of which was the Organization Y (Entity W).'}

--- num_sentences ---
{'de': 16, 'it': 7}

--- sentences ---
{'de': ['Because of their deviation from the Christian faith of the Churc
h, '
        'the canon designates diviners and sorcerers, whether men or wome
n, as '
        'heretics.',
        'The bishops and their helpers, that is, the parish clergy, are to
'
        'fight those who spread the belief in witches and expel those who
are '
        'convicted.',
        'The expulsion of heretics from the parish, as required by the can
on, '
        'is to be regarded as a very severe ecclesiastical penalty.',
        'Obviously particularly serious cases in this legal text are those
of '
        '"rebellious women" who would have turned from God back to Satan a
nd '
        'who would claim or believe that they, along with a large number o
f '
        'other women, were following the pagan goddess Diana in making '
        'long-distance night flights on animals.',
        'These women, according to the canon, would obey the goddess Diana
'
        'like a mistress and be summoned by her on certain nights.',
        'This obedience of the aforementioned women to their mistress Dian
a '
        'and their worship of her as a divine being is interpreted by the
'
        'Canon as unfaithfulness or rebellion against God, apostasy from t
he '
        'true faith and reversion to paganism.',
        'So the fault of these women was not that they were witches, but t
hat '
        'they claimed to be witches when they were not.',
        'The Canon Episcopi is a condemnation of the belief in witchcraf
t.',
        'The Canon Episcopi declares that witchcraft is diabolical.',
        'The key phrase is:',
```

```
        "The idea of women's faith, against which the priests are also to
"
        'preach with all emphasis, is described in the rest of the text as
'
        'imagination and delusion.',
        'Satan, who had transformed himself into an angel of light, would
'
        'project such delusions upon women in order to spread unbelief amo
ng '
        'men.',
        'He could transform himself into the shapes and images of any pers
on '
        'and thus deceive the women in their dreams.',
        'These women would now believe that they had also physically '
        'experienced this illusion of aviation.',
        'So the interpretation of the idea of women from the perspective o
f '
        'early medieval canon law.',
        'It is debatable whether the ideas being challenged conceal a '
        'pagan-Christian belief in fairies that is widespread in Europe in
'
        'various forms.'],
 'it': ['The Canon episcopal defined witchcraft as "worship of the devil",
but '
        'denied that witches could fly physically and declared that "anyon
e '
        'who is so stupid and foolish as to believe such fanciful stories
is '
        'to be considered an infidel, because it is derived from an illusi
on '
        'of the devil".',
        'Such night flights, considered materially impossible and consider
ed '
        'illusory, could however be realized with the spirit and although
'
        'flying with the spirit and imagination, these witches are equally
'
        'guilty, as if they had done it in flesh and blood.',
        'In the 12th century the Canon episcopal created not a few problem
s '
        'for demonologists, because of its internal contradictions: starti
ng '
        'from these deductions, in fact, it was easy to affirm that all '
        'heretics and witches (physically or imagination) had made a pact
with '
        'the devil.',
        'Moreover, it was not possible to defend oneself against possible
'
        'charges, since no one could plausibly prove what he had imagined
or '
        'not.',
        'A different reading of the Canon of Bishops makes it a document o
f '
        'moderation, which reduces witchcraft to pure vanity, execrable bu
t '
        'punishable by disciplinary measures such as expulsion from the '
        'community of believers.',
        'This intuition, which in fact guided the interventions of the bis
hops '
        '(otherwise the civil authorities proceeded), was no longer follow
```

```
ed '
        'by the end of the fifteenth century.',
        'By the middle of the 15th century, the majority of inquisitors an
d '
        "demonologists began to neglect the Bishops' Canon because of its
"
        'problematic nature and preferred to use new and more efficient '
        'inquisitorial manuals, the most widespread of which was the Malle
us '
        'Maleficarum (The Hammer of Witches).']}

--- genres ---
[]

--- similarities_labels ---
['de', 'it']

--- similarities shape ---
(2, 2)
```

Analysis 6: Pipeline Eligibility (EN-only)

The pipeline only includes works with at least two deduplicated English summaries, where English summaries may be either native (from English Wikipedia) or produced by machine translation. Here we provide statistics for the pipeline-eligible subset: total count, the proportion that have a native English summary, and the count that rely solely on translated summaries. We also report how many works are excluded either for having only one English summary or for lacking any translated summaries despite multiple English summaries.

Results:

- There are 29,082 pipeline eligible work, accounting for 98.6% of all works

In [13]:
```
# Pipeline-eligible: EN-only, ≥2 deduped EN summaries, has translated sum
pipeline_eligible_df = story_df[
    (story_df["n_summaries_deduped"] >= 2)
    & (story_df["has_translated_summaries"])
].copy()

n_pipeline_eligible = len(pipeline_eligible_df)
n_total = len(story_df)
n_native_en = int(pipeline_eligible_df["has_en"].sum())
n_trans_only = n_pipeline_eligible - n_native_en
n_singleton_excluded = int((story_df["n_summaries_deduped"] == 1).sum())
n_no_trans_excluded = int(((story_df["n_summaries_deduped"] >= 2) & (~sto


print("\033[1;33m\nPipeline-eligible corpus (EN-only, ≥2 EN deduped, has
print(f"  - Pipeline-eligible works:              {n_pipeline_eligible:>7
print(f"        - with native EN summary (has_en):    {n_native_en:>7,}
print(f"        - translated-only (no native EN):     {n_trans_only:>7,}
print(f"  - Excluded: singletons (n_deduped==1):  {n_singleton_excluded:>
print(f"  - Excluded: ≥2 EN but no translated:    {n_no_trans_excluded:>7
```

```
Pipeline-eligible corpus (EN-only, ≥2 EN deduped, has translated, no anony
mized):
  - Pipeline-eligible works:                29,082  (98.6% of all works)
        - with native EN summary (has_en):     26,384  (90.7% of eligible)
        - translated-only (no native EN):       2,698  (9.3% of eligible)
  - Excluded: singletons (n_deduped==1):      423
  - Excluded: ≥2 EN but no translated:          0
```

Analysis 7: Token Size - Context Window Analysis

This analysis examines the average token size of eligible summaries in our dataset for Llama-3-8B-Instruct. Understanding token distribution helps assess how well our data fits within the model's context window.

Results:

- There are 14 summaries that has more than 8 tokens, these should be excluded (along with their sister stories that belong to same work.)
- Each of these 14 summaries belonging to a different story. Hence, if these stories have multiple summaries available after removal, we can still use them for retrieval task.

In [14]:

```
#Setup: Llama-3-8B-Instruct tokenizer
TOKENIZER_ID = "meta-llama/Meta-Llama-3-8B-Instruct"  # same as Llama-3-8
tokenizer = AutoTokenizer.from_pretrained(TOKENIZER_ID)

# Verify model name
print(f"Using tokenizer from the following model: {tokenizer.name_or_path

def tokenize_and_count(text: str):
    """Tokenize with Llama-3-8B-Instruct tokenizer; return tokens and cou
    if not text or not str(text).strip():
        return [], 0
    enc = tokenizer.encode(str(text), add_special_tokens=False)
    return enc, len(enc)

def token_count(text: str) -> int:
    """Return token count for a single text."""
    return tokenize_and_count(text)[1]

# Get the ids of the pipeline-eligible stories
eligible_ids = set(pipeline_eligible_df["wikidata_id"])

# Tokenize the summaries of the pipeline-eligible stories
records = []
for story in tqdm(storyDataset, total=len(storyDataset), desc="Tokenizing
    if story.wikidata_id not in eligible_ids:
        continue
    ids_en, summaries_en = story.get_all_summaries_en()
    for i, (summary_id, text) in enumerate(zip(ids_en, summaries_en)):
        n_tokens = token_count(text)
        records.append({
            "wikidata_id": story.wikidata_id,
            "summary_id": summary_id,
            "summary_idx": i,
            "n_tokens": n_tokens,
```

```
            "n_chars": len(text),
        })

# Create a dataframe from the records
token_df = pd.DataFrame(records)

# Print the first 5 rows of the dataframe

print("\033[1;33mTable: Token counts and character counts for the first 5
token_df.head(5)
```

```
Using tokenizer from the following model: meta-llama/Meta-Llama-3-8B-Instr
uct
Tokenizing eligible stories: 100%|██████████| 29505/29505 [01:10<00:00, 41
9.64it/s]
Table: Token counts and character counts for the first 5 eligible summarie
s (one row per summary).
```

Out[14]:

	wikidata_id	summary_id	summary_idx	n_tokens	n_chars
0	Q1000352	en	0	1364	6171
1	Q1000352	de	1	743	3292
2	Q1000352	it	2	234	1012
3	Q1000394	en	0	564	2566
4	Q1000394	de	1	204	1064

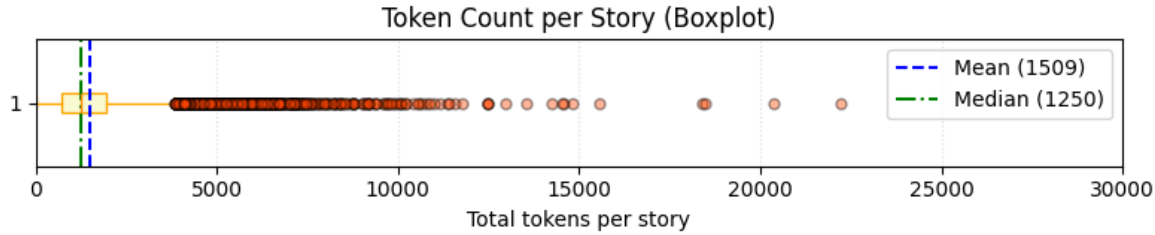
In [15]:

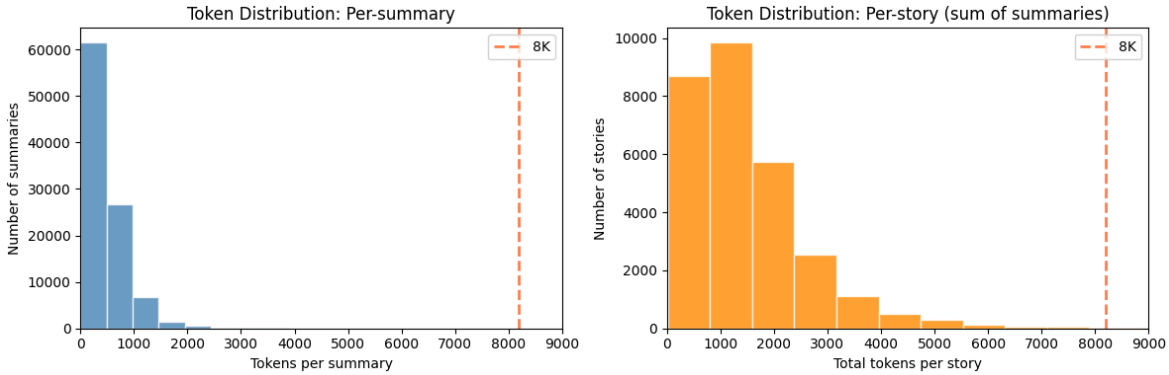
```
# Per-story totals (one value per story)
story_token_stats = token_df.groupby("wikidata_id").agg(
    total_tokens=("n_tokens", "sum"),
).reset_index()
tokens_per_story = story_token_stats["total_tokens"]

# Get the key statistics
mean_val = tokens_per_story.mean()
std_val = tokens_per_story.std()
min_val = tokens_per_story.min()
max_val = tokens_per_story.max()
median_val = tokens_per_story.median()

# Print the key statistics
print("\033[1;33mTokens per story (total per story):\033[0m")
print(f"  Mean   = {mean_val:.2f}")
print(f"  Median = {median_val:.2f}")
print(f"  Std    = {std_val:.2f}")
print(f"  Min    = {min_val:.0f}")
print(f"  Max    = {max_val:.0f}")

# Visualize the key statistics via a boxplot
plt.figure(figsize=(8, 1.8))
box = plt.boxplot(tokens_per_story, vert=False, patch_artist=True,
                  boxprops=dict(facecolor='lightgoldenrodyellow', color='
                  medianprops=dict(color='darkorange'),
                  whiskerprops=dict(color='orange'),
                  capprops=dict(color='orange'),
                  flierprops=dict(markerfacecolor='orangered', marker='o'
plt.title("Token Count per Story (Boxplot)")
plt.xlabel("Total tokens per story")
```

```
plt.grid(axis='x', linestyle=':', alpha=0.4)
plt.axvline(mean_val, color="blue", linestyle="--", label=f"Mean ({mean_v
plt.axvline(median_val, color="green", linestyle="-.", label=f"Median ({m
plt.xlim(0, 30000)
plt.legend()
plt.tight_layout()
plt.show()

# Print the token distributions per summary and per story
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
summary_tokens_xlim = max(9000, token_df["n_tokens"].quantile(0.99))
story_tokens_xlim = max(9000, story_token_stats["total_tokens"].quantile(

# Left: per-summary
axes[0].hist(token_df["n_tokens"], bins=60, color="steelblue", edgecolor=
axes[0].axvline(8192, color="coral", linestyle="--", linewidth=2, label="
axes[0].set_xlabel("Tokens per summary")
axes[0].set_ylabel("Number of summaries")
axes[0].set_title("Token Distribution: Per-summary")
axes[0].set_xlim(0, summary_tokens_xlim)
# Always show legend even if offscreen
axes[0].legend(loc="upper right")

# Right: per-story (total tokens)
axes[1].hist(story_token_stats["total_tokens"], bins=60, color="darkorang
axes[1].axvline(8192, color="coral", linestyle="--", linewidth=2, label="
axes[1].set_xlabel("Total tokens per story")
axes[1].set_ylabel("Number of stories")
axes[1].set_title("Token Distribution: Per-story (sum of summaries)")
axes[1].set_xlim(0, story_tokens_xlim)
axes[1].legend(loc="upper right")

plt.tight_layout()
plt.show()

# Edge Cases: Summaries over 8K
over_8k = token_df[token_df["n_tokens"] > 8192].copy()
n_summaries_over_8k = len(over_8k)
n_stories_over_8k = over_8k["wikidata_id"].nunique()
n_all_summaries = len(token_df)
n_all_stories = token_df["wikidata_id"].nunique()

# Print the edge cases
print("\n\033[1;33mEdge Cases:\033[0m")
print(f"- {n_stories_over_8k} stories ({100 * n_stories_over_8k / n_all_s
      f"have at least one summary exceeding 8K tokens.")
print(f"- {n_summaries_over_8k} summaries ({100 * n_summaries_over_8k / n

# Table: all summaries over 8K (ID = row index)
print("\n\033[1;33mTable: Summaries exceeding 8K tokens (one row per summ
over_8k_display = over_8k[["wikidata_id", "summary_id", "n_tokens"]].sort
over_8k_display = over_8k_display.rename(columns={"summary_id": "language
over_8k_display.index.name = "ID"
display(over_8k_display)
```

Tokens per story (total per story):

```
Mean   = 1509.35
Median = 1250.00
Std    = 1221.97
Min    = 23
Max    = 47196
```





Edge Cases:

- 14 stories (0.05% of all stories) have at least one summary exceeding 8K tokens.
- 14 summaries (0.01% of all summaries) exceed 8K tokens.

Table: Summaries exceeding 8K tokens (one row per summary; sorted by n_tokens descending; ID = row index).

ID	wikidata_id	language	n_tokens
0	Q1216826	es	29241
1	Q374581	de	12439
2	Q2870	es	12130
3	Q123397	de	11387
4	Q492384	de	10816
5	Q161531	de	10804
6	Q478344	de	9815
7	Q183157	fr	9567
8	Q35160	fr	9249
9	Q386431	de	9149
10	Q640513	it	9136
11	Q1211857	de	8813
12	Q131719	fr	8587
13	Q883305	de	8282

Analysis 8: Outlier Summaries/Stories

This cell analyzes stories that contain at least one summary exceeding 8,192 tokens, checking how many of these stories still have enough usable summaries for retrieval after excluding overlong ones.

Results:

- There are 14 stories that each have a single summary exceeding 8K tokens.
- After excluding these long summaries, most stories still have at least two usable summaries for retrieval; specifically, 11 out of 14 remain usable for downstream tasks.

In [16]:
```
# 14 summaries over 8K: confirm they belong to 14 distinct stories
# and show whether each story still has ≥2 summaries for retrieval (using

over_8k = token_df[token_df["n_tokens"] > 8192].copy()
story_ids_over_8k = over_8k["wikidata_id"].unique()
assert len(story_ids_over_8k) == 14, "Expected 14 distinct stories"

# Per story: total summaries, how many over 8K, how many under 8K (usable
rows = []
for wid in story_ids_over_8k:
    sub = token_df[token_df["wikidata_id"] == wid]
    n_total = len(sub)
    n_over_8k = (sub["n_tokens"] > 8192).sum()
    n_under_8k = n_total - n_over_8k
    still_usable = n_under_8k >= 2
```

```
    rows.append({
        "wikidata_id": wid,
        "n_summaries_total": n_total,
        "n_over_8K": n_over_8k,
        "n_under_8K": n_under_8k,
        "still_usable_for_retrieval": still_usable,
    })

table = pd.DataFrame(rows)
print("\033[1;33m14 stories that have at least one summary > 8K tokens:\0
print("- Each has exactly one such summary (14 distinct stories, 14 over-
print("- Still usable for retrieval = after excluding the long summary, ≥
print(f"- Stories still usable for retrieval (≥2 summaries under 8K): {ta
display(table)
```

```
14 stories that have at least one summary > 8K tokens:
- Each has exactly one such summary (14 distinct stories, 14 over-8K summa
ries).
- Still usable for retrieval = after excluding the long summary, ≥2 summar
ies remain.
- Stories still usable for retrieval (≥2 summaries under 8K): 11 / 14 (78.
6%)
```

	wikidata_id	n_summaries_total	n_over_8K	n_under_8K	still_usable_for_retrieva
0	Q1211857	3	1	2	Tru
1	Q1216826	5	1	4	Tru
2	Q123397	2	1	1	Fals
3	Q131719	3	1	2	Tru
4	Q161531	5	1	4	Tru
5	Q183157	3	1	2	Tru
6	Q2870	4	1	3	Tru
7	Q35160	3	1	2	Tru
8	Q374581	2	1	1	Fals
9	Q386431	4	1	3	Tru
10	Q478344	2	1	1	Fals
11	Q492384	3	1	2	Tru
12	Q640513	5	1	4	Tru
13	Q883305	4	1	3	Tru

Analysis 9: Split Representation Analysis

The main purpose here is to ensure that downstream experiments in the English pipeline are valid and that evaluation and training are carried out on appropriate, representative subsets.

This section will:

- Examine which split(s) were used in the original TellMeAgain! dataset and their characteristics.
- Analyze whether the official split is representative for our *pipeline-eligible* (EN-only) subset — i.e., those summaries that would be eligible for our English-only MAVEN pipeline.
- Assess what proportion of the split is actually pipeline-eligible and where non-eligibility arises.
- Explore alternative splitting strategies and compare their characteristics: for example, how many works and summaries are available in each split if we restrict to pipeline-eligible cases, and whether new stratified or random splits might offer advantages.

Results:

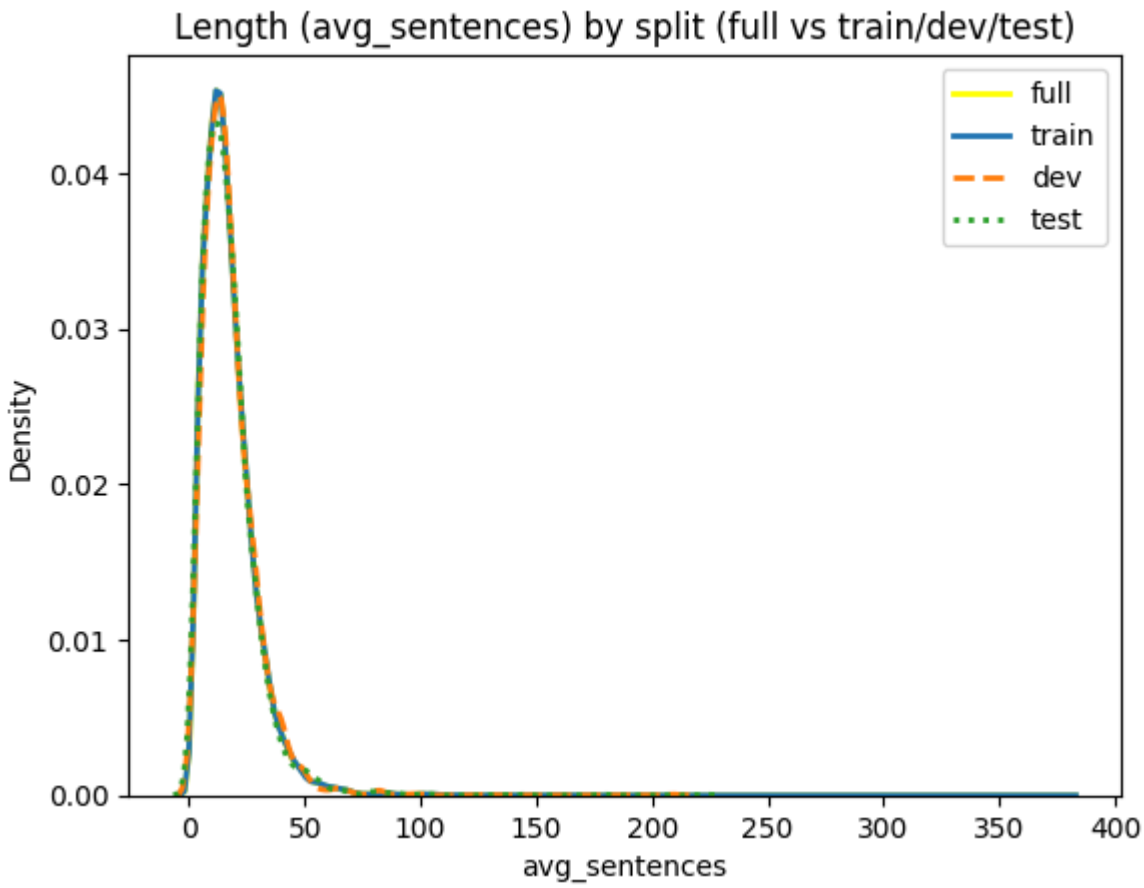
- train and validation (dev) splits represent the full dataset with respect to summary length, number of genres, and proportion of genres by type.

In [17]:
```
# Remove stories with no genre (n_genres == 0)
pipeline_eligible_with_genre_df = pipeline_eligible_df[pipeline_eligible_
print(f"Pipeline-eligible (with genre): {len(pipeline_eligible_with_genre

# Split the dataset into train, dev, and test sets using their official s
split_ids = {}
with zipfile.ZipFile(TELL_ME_AGAIN_ZIP_RAW) as z:
    for name in ["train", "dev", "test"]:
        fname = f"{name}_stories.csv"
        if fname in z.namelist():
            with z.open(fname) as f:
                split_ids[name] = set("Q" + line.strip().decode().strip()


# Map each wikidata_id to its split and attach it to pipeline_eligible_wi
def get_split(wikidata_id):
    if wikidata_id in split_ids.get("train", set()):
        return "train"
    if wikidata_id in split_ids.get("dev", set()):
        return "dev"
    if wikidata_id in split_ids.get("test", set()):
        return "test"
    return "other"
pipeline_eligible_with_genre_df["split"] = pipeline_eligible_with_genre_d
```

```
Pipeline-eligible (with genre): 27986 (dropped 1096 with no genre)
```

In [18]:
```
# Assess the representativeness of the official split based on Length
# Full eligible = pipeline_eligible_with_genre_df; add a "full" label for
df = pipeline_eligible_with_genre_df.copy()
df["subset"] = df["split"].map({"train": "train", "dev": "dev", "test": "

# Summary stats for a length variable (e.g. avg_sentences)
length_col = "avg_sentences"  # or "max_sentences"
stats_full = df[length_col].describe()
stats_train = df.loc[df["split"] == "train", length_col].describe()
stats_dev   = df.loc[df["split"] == "dev", length_col].describe()
pd.DataFrame({"full": stats_full, "train": stats_train, "dev": stats_dev}
```

```
# Plot: full + train, dev, test
sns.kdeplot(data=df, x="avg_sentences", label="full", linewidth=2, linest
for split, ls in [("train", "-"), ("dev", "--"), ("test", ":")]:
    subset = df[df["split"] == split]
    sns.kdeplot(data=subset, x="avg_sentences", label=split, linewidth=2,
plt.legend()
plt.title("Length (avg_sentences) by split (full vs train/dev/test)")
plt.show()
```



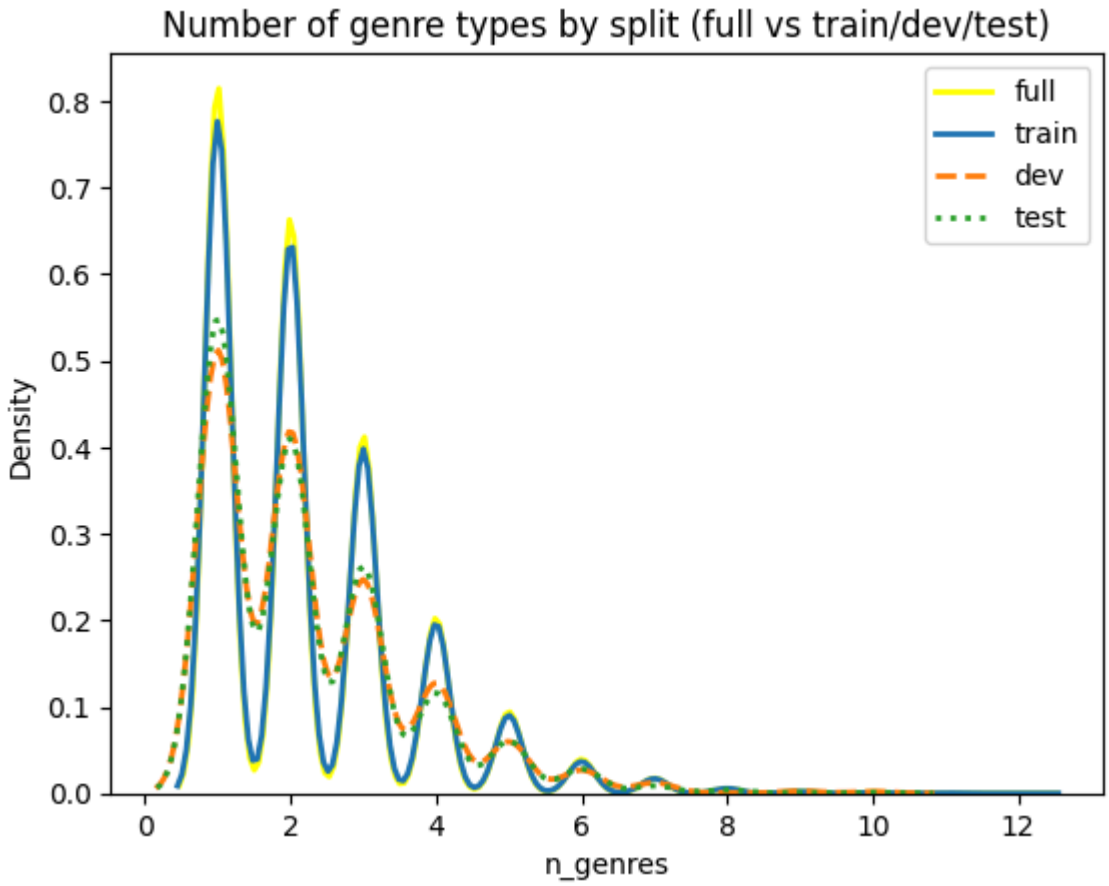
In [19]:

```
# Representativeness by number of genre types (n_genres)
df = pipeline_eligible_with_genre_df.copy()

# Get the statistics for splits
stats_full  = df["n_genres"].describe()
stats_train = df.loc[df["split"] == "train", "n_genres"].describe()
stats_dev   = df.loc[df["split"] == "dev", "n_genres"].describe()
stats_test  = df.loc[df["split"] == "test", "n_genres"].describe()

print("\033[1;33mNumber of genre types by split (full vs train/dev/test):
display(pd.DataFrame({"full": stats_full, "train": stats_train, "dev": st

# Plot: full + train/dev/test
sns.kdeplot(data=df, x="n_genres", label="full", linewidth=2, linestyle="
for split, ls in [("train", "-"), ("dev", "--"), ("test", ":")]:
    subset = df[df["split"] == split]
    sns.kdeplot(data=subset, x="n_genres", label=split, linewidth=2, line
plt.legend()
plt.title("Number of genre types by split (full vs train/dev/test)")
plt.xlabel("n_genres")
plt.show()

# Representativeness by genre types (proportion of each genre within each
```

```
# copy pipeline_eligible_with_genre_df
df = pipeline_eligible_with_genre_df.copy()

def get_genres(wikidata_id):
    """Return the genres for a given Wikidata ID, handling both 'Q'-prefi
    key = wikidata_id[1:] if isinstance(wikidata_id, str) and wikidata_id
    try:
        return storyDataset[key].genres
    except KeyError:
        return []

# map each wikidata_id to its genres
df["genres"] = df["wikidata_id"].map(get_genres)

# explode the genres list and drop rows with empty genres
genre_long = df.explode("genres").dropna(subset=["genres"])[["split", "ge

# Count (split × genre) → proportion within each split
counts = genre_long.groupby(["split", "genres"]).size().unstack(fill_valu
counts = counts.copy()
counts.loc["full"] = counts.sum(axis=0)
props = counts.div(counts.sum(axis=1), axis=0)
top_n = 50
top_genres = props.sum().nlargest(top_n).index
summary = props[top_genres].T.astype(float)
print("\033[1;33mRepresentativeness by genre: proportion within each spli
display(summary)

# Plot: top 15 genres by total proportion within each split
summary.plot(kind="bar", figsize=(12, 5))
plt.ylabel("Proportion within split")
plt.xlabel("Genre (Wikidata Q-ID)")
plt.xticks(rotation=45, ha="right")
plt.legend(title="split")
plt.title("Representativeness by genre: proportion within each split (top
plt.tight_layout()
plt.show()
```

Number of genre types by split (full vs train/dev/test):

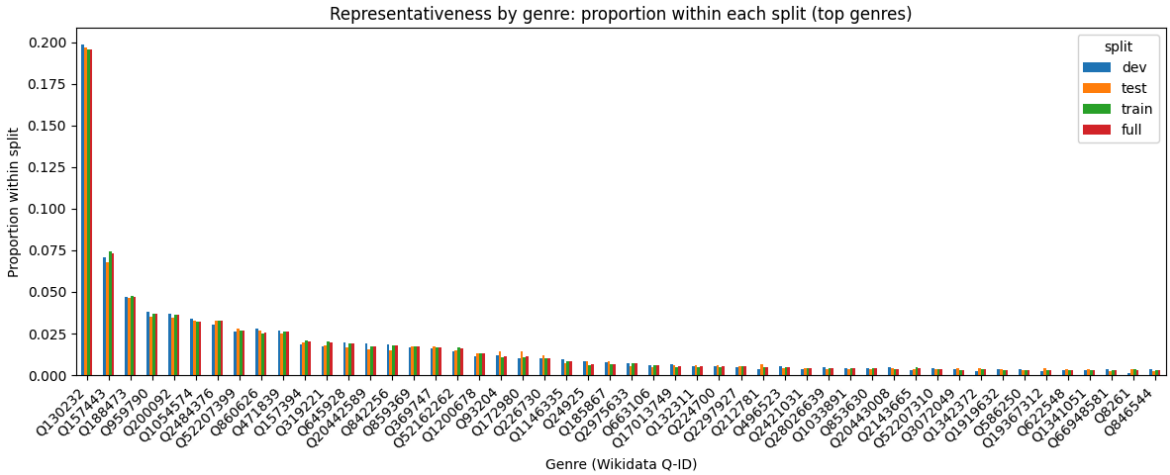
	full	train	dev	test
count	27986.000000	22390.000000	2787.000000	2809.000000
mean	2.262131	2.266994	2.264442	2.221075
std	1.368408	1.367259	1.387322	1.358433
min	1.000000	1.000000	1.000000	1.000000
25%	1.000000	1.000000	1.000000	1.000000
50%	2.000000	2.000000	2.000000	2.000000
75%	3.000000	3.000000	3.000000	3.000000
max	12.000000	12.000000	10.000000	10.000000




```
Representativeness by genre: proportion within each split (top genres):
```

split genres	dev	test	train	full
Q130232	0.198542	0.196987	0.195319	0.195805
Q157443	0.070829	0.067639	0.074491	0.073450
Q188473	0.046744	0.046161	0.047342	0.047166
Q959790	0.037870	0.035262	0.036723	0.036694
Q200092	0.036920	0.034621	0.036349	0.036236
Q1054574	0.034226	0.032698	0.032093	0.032366
Q2484376	0.030265	0.032858	0.032822	0.032571
Q52207399	0.026145	0.028210	0.027030	0.027058
Q860626	0.027729	0.026607	0.025178	0.025573
Q471839	0.027096	0.024844	0.026479	0.026379
Q157394	0.018381	0.019715	0.020824	0.020471
Q319221	0.017588	0.017631	0.020450	0.019887
Q645928	0.019648	0.016509	0.018972	0.018797
Q20442589	0.018856	0.015547	0.017219	0.017217
Q842256	0.018222	0.014906	0.017849	0.017597
Q859369	0.016479	0.017150	0.017455	0.017328
Q369747	0.016162	0.017310	0.016904	0.016870
Q52162262	0.014419	0.014746	0.016411	0.016049
Q1200678	0.011409	0.013303	0.013160	0.013000
Q93204	0.012042	0.014105	0.010993	0.011405
Q172980	0.010141	0.014425	0.011033	0.011278
Q226730	0.010458	0.012021	0.009929	0.010188
Q1146335	0.009666	0.007052	0.008117	0.008166
Q24925	0.008398	0.008495	0.005970	0.006460
Q185867	0.007606	0.008335	0.006423	0.006729
Q2975633	0.007447	0.005129	0.007427	0.007203
Q663106	0.006021	0.004808	0.005950	0.005844
Q17013749	0.006497	0.005770	0.004886	0.005134
Q132311	0.005229	0.006251	0.005103	0.005228
Q224700	0.005546	0.006091	0.004984	0.005149
Q2297927	0.005071	0.005610	0.005319	0.005323
Q212781	0.003644	0.006892	0.004689	0.004802
Q496523	0.005229	0.004167	0.004984	0.004928

split genres	dev	test	train	full
Q2421031	0.003803	0.004488	0.004236	0.004217
Q28026639	0.004754	0.003686	0.004098	0.004123
Q1033891	0.004120	0.003686	0.004354	0.004265
Q853630	0.004120	0.003366	0.004315	0.004202
Q20443008	0.004595	0.004167	0.003527	0.003696
Q2143665	0.003169	0.003526	0.004590	0.004344
Q52207310	0.004120	0.003847	0.003487	0.003586
Q3072049	0.003644	0.004328	0.003073	0.003254
Q1342372	0.002377	0.004167	0.003861	0.003744
Q1919632	0.003644	0.003366	0.003133	0.003207
Q586250	0.003644	0.002885	0.003211	0.003222
Q19367312	0.002535	0.004167	0.002955	0.003033
Q622548	0.003011	0.003526	0.002896	0.002970
Q1341051	0.002852	0.003686	0.002778	0.002875
Q66948581	0.003644	0.002244	0.003172	0.003128
Q8261	0.001426	0.003526	0.003467	0.003270
Q846544	0.003486	0.002244	0.002798	0.002812



Analysis 10: Baseline from Original Distribution

This section establishes reference baselines for our similarity task *without* using any trained or pretrained models. Instead, we focus on baselines derived only from the empirical distribution of similarity scores and class labels in our dataset.

This section will:

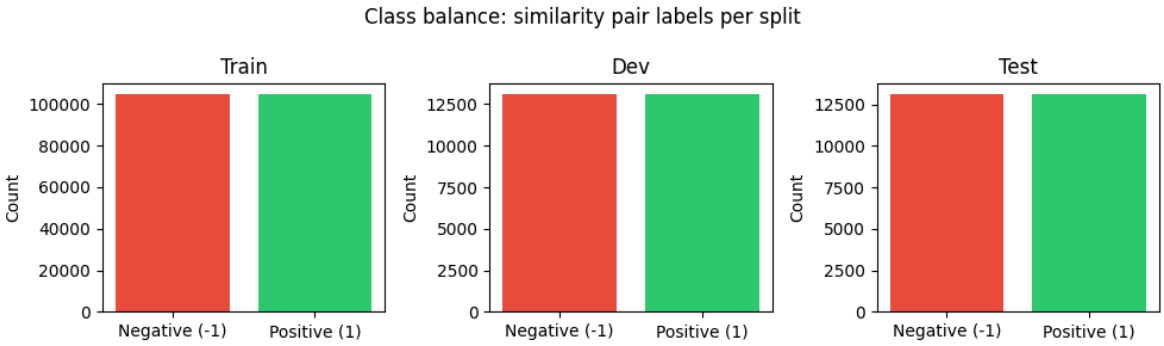
- Compute the distribution of pairwise similarity scores (or class labels, if pairs are already labeled).
- Determine the class balance: the proportion of "positive" (similar) versus "negative" (dissimilar) pairs under a chosen similarity threshold or annotation definition.
- Calculate the accuracy of a simple majority-class predictor, which always guesses the most common class in the data.
- Report the resulting expected baseline accuracy. Any system must exceed this value to demonstrate skill.

Results:

- Class balance: All splits are perfectly balanced: 50% positive (same-story) and 50% negative (different-story) pairs (train: 209,160 pairs; dev: 26,160; test: 26,196).
- Majority baseline accuracy: 50.00% in train, dev, and test. Any similarity model must exceed this to demonstrate skill over the prior.

In [20]:
```
# Baseline from original distribution: class balance and majority-class a
from collections import Counter

print("\033[1;33mBaseline from original distribution (SimilarityDataset)\
for split_name in ["train", "dev", "test"]:
    ds = similarityDataset[split_name]
    labels = [item["label"] for item in ds]
    n_total = len(labels)
    counts = Counter(labels)
    n_pos = counts.get(1, 0)
    n_neg = counts.get(-1, 0)
    prop_pos = n_pos / n_total if n_total else 0
    prop_neg = n_neg / n_total if n_total else 0
    majority_accuracy = max(prop_pos, prop_neg)

    print(f"\n{split_name.upper()}")
    print(f"  Total pairs:     {n_total}")
    print(f"  Positive (1):    {n_pos}  ({prop_pos:.2%})")
    print(f"  Negative (-1):   {n_neg}  ({prop_neg:.2%})")
    print(f"  Majority baseline accuracy: {majority_accuracy:.2%}")


print("\n\033[1;33mAny model must exceed the majority baseline to demonst


# Plot class balance: similarity pair labels per split
fig, axes = plt.subplots(1, 3, figsize=(10, 3))
for ax, split_name in zip(axes, ["train", "dev", "test"]):
    ds = similarityDataset[split_name]
    labels = [item["label"] for item in ds]
    counts = Counter(labels)
    ax.bar(["Negative (-1)", "Positive (1)"], [counts.get(-1, 0), counts.
    ax.set_title(split_name.capitalize())
    ax.set_ylabel("Count")
plt.suptitle("Class balance: similarity pair labels per split")
plt.tight_layout()
plt.show()
```

Baseline from original distribution (SimilarityDataset)

```
TRAIN
  Total pairs:     209160
  Positive (1):    104580  (50.00%)
  Negative (-1):   104580  (50.00%)
  Majority baseline accuracy: 50.00%

DEV
  Total pairs:     26160
  Positive (1):    13080  (50.00%)
  Negative (-1):   13080  (50.00%)
  Majority baseline accuracy: 50.00%

TEST
  Total pairs:     26196
  Positive (1):    13098  (50.00%)
  Negative (-1):   13098  (50.00%)
  Majority baseline accuracy: 50.00%
```

Any model must exceed the majority baseline to demonstrate skill.